



OSTİM TEKNİK ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ BİTİRME PROJESİ RAPORU

API Fortress: REST API Güvenliği Eğitim Simülasyon Laboratuvarı

Proje Ekibi

Mehmet Enes DENİZ
220205039

Ege KILINÇ
220205040

Proje Danışmanı

Dr. Can GÜLDÜREN

Lisans Projesi

Yazılım Mühendisliği
MFBP402 Bitirme Projesi

Mayıs 2026

ONAY

Mehmet Enes DENİZ ve Ege KILINÇ tarafından Dr. Can GÜLDÜREN danışmanlığında hazırlanan “API Fortress: REST API Güvenliği Eğitim Simülasyon Laboratuvarı” başlıklı lisans bitirme projesi tarafımızdan incelenmiş, kapsamı ve niteliği açısından bir Lisans Bitirme Projesi olarak kabul edilmiştir.

Prof. Dr. Jüri Üyesi Adı Soyadı Danışman

Prof. Dr. Jüri Üyesi Adı Soyadı Üye

Doç. Dr. Jüri Üyesi Adı Soyadı Üye

Dr. Öğr. Üyesi Adı Soyadı Bölüm Başkanı

TEŞEKKÜR

Bu çalışmanın her aşamasında değerli yönlendirmeleri ve akademik desteğiyle yanımızda olan danışmanımız Dr. Can GÜLDÜREN'e en derin şükranlarımızı sunarız.

REST API güvenliği üzerine geliştirdiğimiz bu laboratuvar ortamının tasarım ve uygulama sürecinde katkılarını esirgemeyen Yazılım Mühendisliği Bölümü'nün tüm akademisyenlerine teşekkür ederiz.

Proje süresince teknik geri bildirim sağlayan sınıf arkadaşlarımıza ve siber güvenlik konusundaki sorularımıza sabırla yanıt veren topluluklara da minnettarız.

Son olarak, bu süreç boyunca bizi destekleyen ailelerimize sonsuz şükranlarımızı iletiyoruz.

Mehmet Enes DENİZ & Ege KILINÇ

Mayıs 2026, Ankara

ÖZET

Günümüzde REST API yapıları; web uygulamaları, mobil sistemler, mikro servis mimarileri ve bulut tabanlı platformlarda yaygın olarak kullanılmaktadır. API kullanımının artmasıyla birlikte güvenlik açıkları da modern yazılım sistemleri için önemli bir tehdit haline gelmiştir. Özellikle hatalı kimlik doğrulama, yetkilendirme eksiklikleri, IDOR/BOLA zafiyetleri, mass assignment problemleri ve güvenlik kontrollerinin yetersizliği; saldırganların sistem üzerinde yetki yükseltmesine, hassas verilere erişmesine ve kritik işlemleri gerçekleştirmesine neden olabilmektedir. Bu proje kapsamında geliştirilen “API Fortress” sistemi, REST API güvenlik açıklarının uygulamalı olarak öğrenilmesini amaçlayan eğitim tabanlı bir saldırı ve savunma simülasyon laboratuvarı olarak tasarlanmıştır. Sistem, öğrencilerin ve geliştiricilerin güvenlik açıklarını kontrollü bir ortamda deneyimleyebilmesini sağlamak amacıyla iki temel yapıdan oluşmaktadır: güvensiz API ortamı ve güvenli API ortamı. Güvensiz ortamda bilinçli olarak bırakılan zafiyetler üzerinden CTF benzeri hedeflere ulaşılması amaçlanırken, güvenli ortamda aynı açıkların nasıl kapatıldığı ve hangi savunma mekanizmalarıyla engellendiği gösterilmektedir. Proje kapsamında Mass Assignment ile yetki yükseltme, gizli header kullanılarak kimlik doğrulama atlama, parolasız girişe izin veren Broken Authentication mantık hatası, IDOR/BOLA ile başka kullanıcı verilerine erişim ve Broken Function Level Authorization ile yetkisiz kullanıcı silme gibi zafiyet senaryoları uygulanmıştır. Güvenli sürümde ise JWT tabanlı kimlik doğrulama, rol tabanlı yetkilendirme, rate limiting, regex tabanlı Web Application Firewall (WAF), güvenli parola hashleme ve giriş doğrulama kontrolleri geliştirilmiştir. Sistem geliştirme sürecinde Python programlama dili, Flask framework’ü, SQLAlchemy ORM yapısı ve Docker tabanlı çalışma ortamı kullanılmıştır. Gerçekleştirilen testler sonucunda API Fortress’in REST API güvenliği konusunda uygulamalı öğrenme sağlayan işlevsel bir laboratuvar ortamı sunduğu görülmüştür. Bu çalışma, güvenli yazılım geliştirme farkındalığını artırmayı ve API güvenliği alanında saldırı-savunma ilişkisini anlaşılır hale getirmeyi amaçlamaktadır.

Anahtar Kelimeler: REST API, API Güvenliği, OWASP API Security, CTF, Flask, JWT, WAF, IDOR, Mass Assignment, Docker, SQLAlchemy

ABSTRACT

Today, REST API structures are widely used in web applications, mobile systems, microservice architectures, and cloud-based platforms. As API usage increases, security vulnerabilities have become a significant threat to modern software systems. In particular, broken authentication, authorization flaws, IDOR/BOLA vulnerabilities, mass assignment issues, and insufficient security controls may allow attackers to escalate privileges, access sensitive data, and perform critical operations. In this project, the “API Fortress” system was designed as an educational attack and defense simulation laboratory for learning REST API security vulnerabilities in a practical way. The system consists of two main environments: an insecure API environment and a secure API environment. In the insecure environment, intentionally implemented vulnerabilities allow students to reach CTF-like objectives, while the secure environment demonstrates how the same vulnerabilities are mitigated through proper defense mechanisms. Within the scope of the project, several vulnerability scenarios were implemented, including privilege escalation through Mass Assignment, authentication bypass using a hidden header, Broken Authentication caused by missing password validation, IDOR/BOLA for accessing other users’ data, and Broken Function Level Authorization for unauthorized user deletion. In the secure version, JWT-based authentication, role-based authorization, rate limiting, a regex-based Web Application Firewall (WAF), secure password hashing, and input validation mechanisms were developed. Python, the Flask framework, SQLAlchemy ORM, and a Docker-based development environment were used during the implementation process. The tests showed that API Fortress provides a functional laboratory environment for practical REST API security education. This study aims to improve secure software development awareness and make the relationship between attack and defense mechanisms in API security more understandable.

Keywords: REST API, API Security, OWASP API Security, CTF, Flask, JWT, WAF, IDOR, Mass Assignment, Docker, SQLAlchemy

İÇİNDEKİLER

	<u>Sayfa</u>
İÇİNDEKİLER	iv
TABLolar LİSTESİ	v
ŞEKİLLER LİSTESİ	vi
SİMGELER ve KISALTMALAR	vii
1. GİRİŞ	1
1.1. Problem Tanımı	1
1.2. Proje Amaçları	2
1.3. Proje Kapsamı	4
2. REST API GÜVENLİĞİ VE TEMEL KAVRAMLAR	7
2.1. REST API Mimarisi	7
2.2. Kimlik Doğrulama ve Yetkilendirme	7
2.3. OWASP API Security Top 10	7
2.4. Savunma Mekanizmaları	8
3. LİTERATÜR ARAŞTIRMASI VE BENZER SİSTEMLER	9
3.1. Mevcut Eğitim Platformları	9
3.2. Akademik Çalışmalar	11
3.3. API Fortress'in Farklılaştığı Noktalar	14
4. SİSTEM GEREKSİNİMLERİ	17
4.1. Fonksiyonel Gereksinimler	17
4.2. Güvenlik Gereksinimleri	19
4.2.1. Bcrypt algoritması aynı zamanda:	21
4.2.2. Projede temel olarak iki farklı rol bulunmaktadır:	22
4.2.3. RBAC sisteminin temel amacı:	22
4.3. Teknik Gereksinimler	23
4.3.1. ORM kullanımının sağladığı temel avantajlar şunlardır:	24
4.3.2. Bu yaklaşımın temel avantajları şunlardır:	27
5. SİSTEM TASARIMI VE YAZILIM MİMARİSİ	29
5.1. Genel Mimari	29
5.1.1. Sistem temel olarak üç ana katmandan oluşmaktadır:	29
5.1.2. İstemci Katmanı	30
5.1.3. Flask API Katmanı	30
5.1.4. Flask API katmanının temel görevleri şunlardır:	31
5.1.5. Güvenlik Katmanı	31
5.1.6. JWT Authentication	32

5.1.7. Bu yapı sayesinde:	32
5.1.8. RBAC (Role-Based Access Control).....	32
5.1.9. Örneğin:	32
5.1.10. Rate Limiting.....	32
5.1.11. WAF (Web Application Firewall)	33
5.1.12. Örneğin:	33
5.1.13. Veritabanı Katmanı	33
5.1.14. Örneğin:	34
5.1.15. Bu yaklaşım:	34
5.1.16. REST API İletişim Yapısı	34
5.1.17. Docker Tabanlı Çalışma Ortamı	35
5.1.18. Docker mimarisi sayesinde:	35
5.2. Modüler Backend Yapısı	35
5.2.1. insecure_api Klasörü	36
5.2.2. secure_api Klasörü	37
5.2.3. common Klasörü	38
5.2.4. docker-compose.yml Dosyası	38
5.2.5. requirements.txt Dosyası	39
5.2.6. Blueprint Mimarisi	39
5.3. Veritabanı Tasarımı	40
5.4. REST API Endpoint Yapısı	41
6. GÜVENSİZ API ORTAMI VE ZAFİYET SENARYOLARI.....	42
6.1. Mass Assignment	42
6.1.1. Bu zafiyet özelliikle:	44
6.1.2. Bu senaryo sayesinde kullanıcılar:	44
6.2. Broken Authentication	44
6.2.1. Bu yapı gerçek sistemlerde zaman zaman:	45
6.2.2. Bu zafiyetin temel amacı:	46
6.2.3. Broken Authentication problemleri özelliikle:	46
6.3. IDOR / BOLA	46
6.3.1. Bu endpoint üzerinde:	47
6.3.2. Bu saldırı tipi özelliikle:	47
6.3.3. Bu senaryo sayesinde kullanıcılar:	48
6.4. Broken Function Level Authorization	48
6.4.1. Bu problem genellikle:	48
6.4.2. Bu endpoint üzerinde:	49

6.4.3. Bu senaryo sayesinde kullanıcılar:	50
7. GÜVENLİ API ORTAMI VE SAVUNMA MEKANİZMALARI	51
7.1. JWT Tabanlı Kimlik Doğrulama	51
7.1.1. JWT doğrulama mekanizması sayesinde:	52
7.1.2. Bu mekanizma özellikle:	52
7.2. Rol Tabanlı Yetkilendirme (RBAC)	53
7.2.1. Örneğin:	53
7.2.2. Bu yapı özellikle:	53
7.2.3. Bu yapıda:	54
7.3. Rate Limiting	54
7.3.1. Rate limiting mekanizmasının temel amacı:	54
7.3.2. Gerçek dünya sistemlerinde rate limiting:	55
7.4. Web Application Firewall (WAF)	55
7.4.1. WAF sisteminin temel amacı:	55
7.4.2. Bu pattern'ler:	56
7.4.3. Amaç:	57
7.5. Güvenli Parola Hashleme ve Input Validation	57
7.5.1. Bcrypt algoritmasının kullanılmasının temel amacı:	57
7.5.2. Bu işlem sonucunda:	57
7.5.3. Bu yapı sayesinde:	58
7.5.4. Örneğin:	58
7.5.5. Validation mekanizmasının temel amacı:	58
7.5.6. Bu yaklaşım:	59
8. UYGULAMA GELİŞTİRME SÜRECİ	60
8.1. Geliştirme Ortamı	60
8.2. Docker Konteyner Yapısı	60
8.3. Kod Organizasyonu	61
8.3.1. Güvenli CORS yapılandırmasının amacı şudur:	64
9. TEST SÜRECİ VE CTF SENARYOLARI	65
9.1. Test Stratejisi	65
9.1.1. Test sürecinde üç temel yöntem kullanılmıştır:	65
9.1.2. Vulnerable API ortamında gerçekleştirilen testlerde:	66
9.1.3. Secure API ortamında yapılan testlerde ise:	66
9.2. Zafiyet Testleri	66
9.3. Test Sonuçları	67
10.SONUÇLAR VE ÖNERİLER	69

10.1.Gelecek Çalışmalar	71
11.Ek-1: Kod Gösterimi	75
11.1.app.py – Güvenli API Başlangıç Noktası	75
11.2.models.py – Kullanıcı Modeli	75

TABLÖLAR LİSTESİ

Sayfa

ŞEKİLLER LİSTESİ

Sayfa

SİMGELELER ve KISALTMALAR

API	: Application Programming Interface (Uygulama Programlama Arayüzü)
BOLA	: Broken Object Level Authorization
CTF	: Capture The Flag
CRUD	: Create, Read, Update, Delete
Docker	: Konteyner tabanlı sanallaştırma platformu
DVWA	: Damn Vulnerable Web Application
HTTP	: HyperText Transfer Protocol
IDOR	: Insecure Direct Object Reference
JWT	: JSON Web Token
JSON	: JavaScript Object Notation
MVP	: Minimum Viable Product (Minimum Uygulanabilir Ürün)
OWASP	: Open Web Application Security Project
ORM	: Object Relational Mapping
RBAC	: Role-Based Access Control (Rol Tabanlı Erişim Kontrolü)
REST	: Representational State Transfer
SQL	: Structured Query Language
WAF	: Web Application Firewall (Web Uygulama Güvenlik Duvarı)
XSS	: Cross-Site Scripting

1. GİRİŞ

1.1. Problem Tanımı

Modern web tabanlı uygulamaların büyük çoğunluğu, istemci ve sunucu arasındaki veri iletişimini REST API mimarisi üzerinden gerçekleştirmektedir. Günümüzde mobil uygulamalar, mikro servis sistemleri, IoT platformları, bulut tabanlı servisler ve modern web uygulamaları; veri alışverişi, kullanıcı doğrulama, servis entegrasyonu ve sistemler arası haberleşme işlemleri için yoğun biçimde API teknolojilerine ihtiyaç duymaktadır. REST mimarisi; hafif yapısı, platform bağımsız çalışabilmesi, HTTP protokolü ile uyumlu olması ve ölçeklenebilirliği nedeniyle günümüzde en yaygın kullanılan servis mimarilerinden biri haline gelmiştir. Özellikle frontend ve backend katmanlarının birbirinden ayrıldığı modern yazılım mimarilerinde REST API'ler, sistemin temel iletişim omurgasını oluşturmaktadır.

REST API kullanımının yaygınlaşmasıyla birlikte güvenlik problemleri de önemli ölçüde artmıştır. API'ler doğrudan internet üzerinden erişilebilir olduğu için saldırganlar açısından önemli hedefler haline gelmiştir. Yetkisiz erişim, SQL Injection saldırıları, Cross-Site Scripting (XSS), brute force giriş denemeleri, token manipülasyonu, Broken Authentication, güvenlik doğrulama eksiklikleri, hatalı yetkilendirme mekanizmaları ve servis yoğunlaştırma (DoS/DDoS) saldırıları gibi tehditler; güvenli olmayan API sistemlerinde ciddi veri kayıplarına, kullanıcı bilgilerinin sızdırılmasına, servis kesintilerine ve sistem bütünlüğünün bozulmasına neden olabilmektedir. Özellikle kullanıcı doğrulama ve veri erişim kontrollerinin yanlış yapılandırıldığı API sistemlerinde saldırganlar, sistem içerisindeki hassas verilere erişebilmekte veya sistem davranışını manipüle edebilmektedir.

OWASP (Open Web Application Security Project) tarafından yayınlanan "API Security Top 10" listesi, modern API sistemlerinde en sık karşılaşılan güvenlik açıklarını ve bu açıkların ne kadar kritik sonuçlar doğurabileceğini açıkça göstermektedir. Bu listede yer alan Broken Object Level Authorization (BOLA), Broken Authentication, Excessive Data Exposure, Security Misconfiguration ve Injection saldırıları; günümüzde birçok gerçek sistemde karşılaşılan temel güvenlik problemleri arasında yer almaktadır. Özellikle hızlı geliştirme süreçlerinde güvenlik kontrollerinin ikinci plana atılması, geliştiricilerin yalnızca işlevselliğe odaklanması ve güvenlik testlerinin yeterince yapılmaması; REST API tabanlı sistemlerde ciddi zafiyetlerin oluşmasına neden olmaktadır.

Bununla birlikte REST API güvenliği konusunda mevcut eğitim materyallerinin büyük çoğunluğu teorik düzeyde kalmakta; geliştiricilerin gerçek saldırı ve savunma

mekanizmalarını deneyimleyebileceği kontrollü laboratuvar ortamları yeterince yaygın bulunmamaktadır. Birçok eğitim içeriği yalnızca kavramsal açıklamalar sunmakta, ancak güvenlik açıklarının uygulama üzerinde nasıl oluştuğunu, saldırıların sisteme nasıl etki ettiğini ve bu saldırılara karşı hangi koruma mekanizmalarının nasıl çalıştığını pratik olarak göstermemektedir. Bu durum, güvenli yazılım geliştirme farkındalığının artırılmasında önemli bir eksikliğe yol açmaktadır.

Ayrıca geliştiricilerin önemli bir kısmı, REST API güvenliğini yalnızca kullanıcı giriş sistemi veya token doğrulama mekanizması olarak değerlendirmekte; input validation, rate limiting, güvenli hata yönetimi, erişim kontrolü, veri filtreleme ve saldırı tespit mekanizmaları gibi kritik güvenlik katmanlarını yeterince uygulamamaktadır. Özellikle küçük ve orta ölçekli projelerde güvenlik önlemleri çoğu zaman proje geliştirme sürecinin son aşamasına bırakılmakta veya tamamen göz ardı edilmektedir. Bu durum, saldırganların sistemlere çok daha kolay erişebilmesine zemin hazırlamaktadır.

Bu proje kapsamında geliştirilen güvenli REST API laboratuvar sistemi ile; hem modern REST API mimarisinin temel çalışma prensiplerinin gösterilmesi hem de yaygın güvenlik açıklarının kontrollü bir ortamda analiz edilmesi amaçlanmıştır. Sistem içerisinde kullanıcı doğrulama mekanizmaları, modüler API yapısı, rate limiting, regex tabanlı temel WAF kontrolleri, CORS güvenliği ve saldırı filtreleme mekanizmaları uygulanarak güvenli backend geliştirme yaklaşımı örneklenmiştir. Böylece geliştiricilerin yalnızca teorik bilgi edinmesi değil, aynı zamanda gerçekçi saldırı senaryolarını ve savunma mekanizmalarını uygulamalı olarak deneyimleyebilmesi hedeflenmiştir.

Bu çalışma aynı zamanda güvenli yazılım geliştirme kültürünün yaygınlaştırılmasına katkı sağlamayı amaçlamaktadır. Projede geliştirilen yapı sayesinde geliştiriciler; REST API sistemlerinde oluşabilecek güvenlik açıklarını analiz edebilmekte, saldırı yüzeylerini gözlemleyebilmekte ve modern backend mimarilerinde uygulanması gereken temel güvenlik prensiplerini pratik olarak inceleyebilmektedir. Böylece proje, hem eğitimsel hem de teknik açıdan REST API güvenliği konusunda uygulamalı bir öğrenme ortamı sunmaktadır.

1.2. Proje Amaçları

Bu çalışmanın temel amacı, REST API güvenlik açıklarının teorik bilgilerden öte uygulamalı olarak öğrenilmesini sağlayan eğitim tabanlı bir saldırı ve savunma simülasyon laboratuvarı geliştirmektir. Günümüzde REST API teknolojileri modern yazılım sistemlerinin temel bileşenlerinden biri haline gelmiş olsa da API güvenliği konusunda

uygulamalı eğitim ortamlarının yetersiz olması, geliştiricilerin gerçek saldırı senaryolarını yeterince deneyimleyememesine neden olmaktadır. Bu proje ile birlikte hem saldırı tekniklerinin hem de modern savunma mekanizmalarının kontrollü bir laboratuvar ortamında deneyimlenebilmesi hedeflenmiştir.

Çalışmanın bir diğer amacı, güvenli yazılım geliştirme süreçlerinde karşılaşılan yaygın güvenlik problemlerinin gerçek sistem mimarileri üzerinde analiz edilmesini sağlamaktır. Bu kapsamda geliştirilen sistem yalnızca teorik bilgi sunan bir eğitim platformu değil; aynı zamanda kullanıcıların güvenlik açıklarını keşfedebildiği, saldırı mantığını anlayabildiği ve savunma mekanizmalarının sistem davranışını nasıl değiştirdiğini gözlemleyebildiği uygulamalı bir siber güvenlik laboratuvarı olarak tasarlanmıştır.

Bu proje kapsamında OWASP tarafından yayınlanan API Security Top 10 listesindeki kritik güvenlik açıklarının gerçek bir sistem üzerinde simüle edilmesi amaçlanmıştır. Özellikle Broken Authentication, Broken Object Level Authorization (BOLA), Injection saldırıları, Security Misconfiguration, Excessive Data Exposure ve Rate Limiting eksiklikleri gibi yaygın API zafiyetlerinin hem saldırı hem de savunma perspektifinden incelenmesi hedeflenmiştir. Böylece kullanıcıların yalnızca güvenlik açığının teorik tanımını öğrenmesi değil; bu açığın sistem içerisinde nasıl oluştuğunu, saldırgan tarafından nasıl kullanılabildiğini ve hangi savunma yöntemleriyle engellenebileceğini uygulamalı olarak deneyimlemesi amaçlanmıştır.

Projenin önemli hedeflerinden biri de öğrencilere ve geliştiricilere CTF (Capture The Flag) mantığına benzer etkileşimli bir öğrenme deneyimi sunmaktır. Bu yapı sayesinde kullanıcılar yalnızca pasif şekilde eğitim materyali okumak yerine; gerçek saldırı senaryolarını çözmeye çalışarak sistem güvenliği konusunda aktif deneyim kazanabilmektedir. Özellikle güvenlik açıklarının kontrollü görevler halinde sunulması, kullanıcıların saldırı mantığını adım adım analiz edebilmesine olanak sağlamaktadır. Böylece öğrenme süreci daha interaktif, kalıcı ve uygulama odaklı hale getirilmektedir.

Çalışmanın bir diğer temel amacı, aynı sistem üzerinde hem güvensiz (vulnerable) hem de güvenli (secure) API ortamları oluşturarak karşılaştırmalı analiz yapılabilmesini sağlamaktır. Güvensiz ortamda kullanıcılar çeşitli güvenlik açıklarını deneyimleyebilirken; güvenli ortamda aynı saldırıların neden başarısız olduğu gözlemlenebilmektedir. Bu yaklaşım sayesinde modern savunma mekanizmalarının sistem güvenliği üzerindeki etkisi doğrudan analiz edilebilmektedir. Özellikle güvenlik katmanlarının etkinliğini uygulamalı olarak görmek, geliştiricilerin güvenli backend mimarisi konusundaki farkındalığını artırmaktadır.

Proje kapsamında modern API güvenlik mekanizmalarının uygulamalı olarak gösterilmesi de hedeflenmiştir. Bu doğrultuda JWT (JSON Web Token) tabanlı kimlik doğrulama sistemi, rol tabanlı erişim kontrolü (RBAC), regex tabanlı temel WAF (Web Application Firewall) filtreleme mekanizması, rate limiting sistemi ve güvenli yapılandırma kontrolleri sisteme entegre edilmiştir. Böylece kullanıcılar yalnızca saldırı tekniklerini değil; aynı zamanda modern backend sistemlerinde kullanılan profesyonel savunma yöntemlerini de inceleyebilmektedir. Özellikle JWT yapısının çalışma mantığı, token doğrulama süreçleri, erişim kontrol mekanizmaları ve istek filtreleme sistemleri proje içerisinde uygulamalı olarak gösterilmektedir.

Bu çalışmanın eğitimsel hedeflerinden biri de güvenli yazılım geliştirme kültürünün yaygınlaştırılmasına katkı sağlamaktır. Günümüzde birçok yazılım projesinde güvenlik önlemleri geliştirme sürecinin son aşamasında ele alınmakta veya tamamen göz ardı edilmektedir. Bu proje ile birlikte güvenliğin yazılım geliştirme sürecinin merkezinde yer alması gerektiği vurgulanmaktadır. Özellikle geliştiricilerin daha proje tasarım aşamasında güvenlik odaklı düşünme alışkanlığı kazanması hedeflenmiştir.

Ayrıca proje, siber güvenlik eğitimi alan öğrenciler için uygulamalı bir laboratuvar ortamı sunmayı amaçlamaktadır. Kullanıcılar bu sistem sayesinde API güvenliği, kimlik doğrulama mekanizmaları, erişim kontrol sistemleri, saldırı tespit yöntemleri ve güvenlik filtreleme teknikleri gibi konuları gerçekçi bir backend mimarisi üzerinde deneyimleyebilmektedir. Böylece proje yalnızca teknik bir yazılım geliştirme çalışması değil; aynı zamanda eğitim ve farkındalık odaklı bir siber güvenlik platformu niteliği taşımaktadır.

Sonuç olarak bu çalışmanın temel amacı; REST API güvenliği konusunda teorik bilgi ile pratik uygulama arasındaki boşluğu azaltmak, geliştiricilerin gerçek dünya saldırı senaryolarını kontrollü şekilde deneyimleyebilmesini sağlamak ve modern güvenlik mekanizmalarının çalışma mantığını uygulamalı olarak öğretmektir. Bu sayede proje, hem yazılım geliştirme hem de siber güvenlik eğitimi açısından kapsamlı bir öğrenme ve analiz ortamı sunmaktadır.

1.3. Proje Kapsamı

Bu proje, modern REST API güvenlik yaklaşımlarının uygulamalı olarak analiz edilebilmesi amacıyla geliştirilmiş, Docker tabanlı modüler bir siber güvenlik laboratuvar sistemidir. Sistem, aynı konteyner ortamı içerisinde çalışan ve birbirinden bağımsız şekilde yapılandırılmış iki ayrı REST API backend yapısından oluşmaktadır. Bu backend'lerden ilki

kasıtlı olarak güvenlik açıkları içeren “güvensiz API ortamını (vulnerable environment)”, ikinci backend ise modern savunma mekanizmalarıyla güçlendirilmiş “güvenli API ortamını (secure environment)” temsil etmektedir. Böylece kullanıcılar aynı işlevleri yerine getiren iki farklı API mimarisini karşılaştırmalı olarak inceleyebilmekte ve güvenlik mekanizmalarının sistem davranışı üzerindeki etkisini uygulamalı biçimde gözlemleyebilmektedir.

Proje kapsamında geliştirilen her iki backend sistemi de gerçek dünya REST API mimarisine benzer şekilde tasarlanmıştır. Sistemlerde kullanıcı yönetimi, kullanıcı kayıt ve giriş işlemleri, JWT tabanlı kimlik doğrulama, rol tabanlı erişim kontrolü (RBAC), veri listeleme, veri oluşturma, güncelleme ve silme gibi temel API işlemleri bulunmaktadır. Böylece proje yalnızca teorik saldırı örnekleri sunan basit bir demo sistemi olmaktan çıkarılarak gerçekçi backend mimarisine yakın bir laboratuvar ortamı haline getirilmiştir.

Güvensiz API ortamında çeşitli güvenlik açıkları bilinçli şekilde sisteme entegre edilmiştir. Bu ortamda kullanıcılar SQL Injection, zayıf kimlik doğrulama mekanizmaları, eksik erişim kontrolü, yetersiz input validation, Broken Object Level Authorization (BOLA), hatalı güvenlik yapılandırmaları ve rate limiting eksiklikleri gibi yaygın REST API zafiyetlerini analiz edebilmektedir. Bu yapı sayesinde saldırgan bakış açısıyla sistem davranışı incelenebilmekte ve güvenlik açıklarının oluşma nedenleri uygulamalı olarak gözlemlenebilmektedir.

Güvenli API ortamında ise aynı işlevler modern güvenlik prensiplerine uygun şekilde yeniden yapılandırılmıştır. Bu ortamda JWT tabanlı güvenli kimlik doğrulama sistemi, rol tabanlı erişim kontrolü, regex tabanlı temel WAF (Web Application Firewall) filtreleme sistemi, rate limiting mekanizması, güvenli input validation işlemleri, güvenli hata yönetimi ve CORS güvenlik yapılandırmaları uygulanmıştır. Böylece kullanıcılar yalnızca saldırıların nasıl gerçekleştiğini değil; aynı zamanda bu saldırıların hangi savunma yöntemleriyle engellenebileceğini de doğrudan analiz edebilmektedir.

Proje kapsamında Docker teknolojisi kullanılarak tüm sistemin konteyner tabanlı çalışması sağlanmıştır. Bu yaklaşım sayesinde hem güvenli hem de güvensiz backend servisleri izole şekilde çalıştırılabilmekte, sistemin kurulumu kolaylaşmakta ve laboratuvar ortamının taşınabilirliği artırılmaktadır. Docker kullanımı aynı zamanda gerçek dünya mikro servis mimarilerine benzer bir geliştirme ortamı sunmaktadır. Böylece kullanıcılar yalnızca API güvenliği değil, modern backend dağıtım mantığı hakkında da deneyim kazanabilmektedir.

Sistem içerisinde frontend tarafı temel düzeyde tutulmuş, ana odak backend güvenliği ve API davranışlarının analiz edilmesine verilmiştir. Bu nedenle proje kapsamı içerisinde

gelişmiş kullanıcı arayüzü geliştirme süreçleri yer almamaktadır. Kullanıcı etkileşimleri ağırlıklı olarak REST istemcileri, HTTP istekleri ve API endpoint testleri üzerinden gerçekleştirilmektedir. Böylece dikkat doğrudan güvenlik mekanizmaları ve backend davranışlarına yönlendirilmiştir.

Proje kapsamında özellikle uygulama katmanı güvenliği hedeflenmiş, ağ seviyesindeki gelişmiş saldırı senaryoları kapsam dışında bırakılmıştır. Distributed Denial of Service (DDoS), Man-in-the-Middle (MITM), DNS spoofing, packet sniffing ve düşük seviyeli ağ saldırıları gibi konular bu çalışmanın sınırları dışında tutulmuştur. Bunun temel nedeni, çalışmanın odak noktasının ağ güvenliği değil REST API uygulama güvenliği olmasıdır. Ayrıca bu tür saldırıların analiz edilebilmesi için daha karmaşık ağ topolojileri, sanal ağ yapılandırmaları ve gelişmiş güvenlik altyapıları gerekmektedir.

Bununla birlikte proje kapsamında makine öğrenimi tabanlı saldırı tespit sistemleri veya anomali analiz mekanizmaları da yer almamaktadır. Günümüzde birçok modern güvenlik sistemi yapay zekâ destekli saldırı tespiti, trafik analizi ve davranış modelleme yöntemleri kullanmasına rağmen bu çalışma temel olarak REST API güvenlik mimarisi ve uygulama katmanı savunmaları üzerine odaklanmıştır. Ancak ilerleyen çalışmalarda sisteme yapay zekâ tabanlı anomali tespiti, saldırı davranışı analizi ve otomatik tehdit sınıflandırma mekanizmalarının eklenmesi planlanmaktadır.

Ayrıca proje kapsamı içerisinde gerçek üretim ortamlarında kullanılan gelişmiş güvenlik servisleri, merkezi log yönetimi, SIEM entegrasyonları, dağıtık güvenlik izleme sistemleri ve kurumsal seviyede API gateway çözümleri yer almamaktadır. Bu sistemler gelecekte projenin daha ileri sürümlerinde genişletilebilir modüller olarak değerlendirilmektedir.

Sonuç olarak bu proje; REST API güvenliğinin uygulamalı olarak öğrenilebilmesi amacıyla geliştirilmiş, güvenli ve güvensiz backend sistemlerini karşılaştırmalı şekilde sunan Docker tabanlı bir eğitim ve analiz laboratuvarı niteliğindedir. Proje kapsamında temel odak uygulama katmanı güvenliği, saldırı-senaryo analizi ve modern savunma mekanizmalarının incelenmesi olurken; gelişmiş ağ saldırıları, frontend odaklı kullanıcı deneyimi geliştirmeleri ve yapay zekâ destekli güvenlik sistemleri kapsam dışında bırakılmıştır. Böylece proje, kontrollü ve anlaşılabilir bir laboratuvar ortamında REST API güvenlik farkındalığını artırmayı amaçlayan kapsamlı bir uygulama platformu sunmaktadır.

2. REST API GÜVENLİĞİ VE TEMEL KAVRAMLAR

2.1. REST API Mimarisi

REST (Representational State Transfer), istemci ve sunucu arasındaki veri iletişimini HTTP protokolü üzerinden gerçekleştiren yazılım mimarisidir. REST tabanlı sistemlerde veri alışverişi genellikle JSON (JavaScript Object Notation) formatı üzerinden sağlanmaktadır. GET, POST, PUT ve DELETE gibi HTTP metodları kullanılarak istemci ile sunucu arasında kaynak odaklı veri aktarımı gerçekleştirilmektedir.

REST API yapısının temel ilkeleri şunlardır: durumsuzluk (statelessness), istemci-sunucu ayrımı, önbelleğe alınabilirlik (cacheability), katmanlı sistem (layered system) ve tek tip arayüz (uniform interface). Bu ilkeler, ölçeklenebilir ve sürdürülebilir sistem tasarımı mümkün kılmaktadır.

2.2. Kimlik Doğrulama ve Yetkilendirme

Kimlik doğrulama (authentication), sisteme giriş yapan kullanıcının gerçekten belirtilen kullanıcı olup olmadığının kontrol edilmesini ifade etmektedir. Yetkilendirme (authorization) ise doğrulanan kullanıcının hangi işlemleri gerçekleştirebileceğinin belirlenmesini sağlamaktadır. Modern API sistemlerinde bu işlemler çoğunlukla JWT (JSON Web Token) tabanlı yapılar ile gerçekleştirilmektedir.

JWT, üç bölümden oluşan imzalı bir token yapısıdır: header (algoritma bilgisi), payload (kullanıcı verileri) ve signature (imza). Kullanıcı başarıyla giriş yaptığında sunucu tarafından bir JWT üretilir ve bu token istemci tarafında saklanarak her istekte Authorization başlığı ile sunucuya iletilir. Sunucu, token imzasını doğrulayarak kullanıcının kimliğini ve yetkilerini belirler.

2.3. OWASP API Security Top 10

OWASP API Security Top 10, REST API sistemlerinde en yaygın görülen güvenlik açıklarını tanımlayan güvenlik standardıdır. Bu proje kapsamında ele alınan başlıca zafiyetler şunlardır:

- Broken Object Level Authorization (BOLA/IDOR): Kullanıcıların başka kullanıcılara ait verilere erişebilmesi.
- Broken Authentication: Kimlik doğrulama mekanizmalarının yetersiz veya hatalı uygulanması.
- Broken Function Level Authorization: Normal kullanıcıların yönetici işlevlerine erişebilmesi.
- Mass Assignment: İstemci tarafından gönderilen verilerin filtrelenmeden doğrudan işlenmesi.
- Security Misconfiguration: Güvenlik ayarlarının yanlış veya eksik yapılandırılması.

2.4. Savunma Mekanizmaları

Modern API güvenliği yaklaşımlarında rate limiting, input validation ve Web Application Firewall (WAF) sistemleri önemli savunma mekanizmaları arasında yer almaktadır. Rate limiting mekanizması, istemcilerin belirli süre içerisinde gerçekleştirebileceği istek sayısını sınırlandırırken; WAF sistemleri, zararlı istekleri filtreleyerek saldırı girişimlerini engellemeyi amaçlamaktadır. Güvenli parola hashleme ve katı giriş doğrulama kontrolleri de saldırı yüzeyini önemli ölçüde azaltan temel güvenlik önlemleri arasında sayılabilir.

3. LİTERATÜR ARAŞTIRMASI VE BENZER SİSTEMLER

3.1. Mevcut Eğitim Platformları

REST API güvenliği ve web uygulama güvenliği konusunda geliştirilen eğitim platformları, son yıllarda siber güvenlik eğitim süreçlerinde önemli bir yere sahip olmuştur. Özellikle uygulamalı öğrenme yaklaşımının yaygınlaşmasıyla birlikte kullanıcıların gerçek güvenlik açıklarını kontrollü laboratuvar ortamlarında deneyimleyebilmesine olanak sağlayan platformlar geliştirilmiştir. Bu platformlar sayesinde öğrenciler, geliştiriciler ve siber güvenlik araştırmacıları yalnızca teorik bilgi edinmekle kalmamakta; aynı zamanda saldırı tekniklerini, güvenlik açıklarının oluşma mantığını ve temel savunma yöntemlerini uygulamalı olarak inceleyebilmektedir.

Bu alandaki en yaygın ve öncü platformların başında OWASP Juice Shop, DVWA (Damn Vulnerable Web Application) ve PortSwigger Web Security Academy gelmektedir. Bu platformlar güvenlik eğitimi açısından önemli katkılar sağlamasına rağmen, modern REST API güvenliği konusunda bazı sınırlılıklara sahiptir.

OWASP tarafından geliştirilen OWASP Juice Shop, bilinçli olarak güvenlik açıkları içeren modern web uygulamalarından biridir. Platform, gerçek bir e-ticaret sistemi mantığıyla tasarlanmış olup kullanıcı giriş sistemi, ürün yönetimi, alışveriş işlemleri ve veri işleme gibi birçok gerçek dünya senaryosunu içermektedir. Sistem içerisinde authentication bypass, Broken Access Control, SQL Injection, Cross-Site Scripting (XSS), güvenlik yapılandırma hataları ve hassas veri sızıntıları gibi birçok farklı güvenlik açığı bulunmaktadır. Kullanıcılar bu açıkları çözerek sistemdeki “challenge” görevlerini tamamlamaktadır. Juice Shop’un en önemli avantajlarından biri modern web teknolojileri kullanılarak geliştirilmiş olması ve kullanıcıya gerçekçi saldırı senaryoları sunabilmesidir.

Bununla birlikte OWASP Juice Shop ağırlıklı olarak genel web uygulama güvenliği üzerine odaklanmaktadır. Platform içerisinde bazı API tabanlı senaryolar bulunsada sistemin temel amacı doğrudan REST API güvenliğini karşılaştırmalı olarak analiz etmek değildir. Ayrıca platform içerisinde güvenli ve güvensiz API ortamlarının aynı anda sunulmaması, kullanıcıların saldırı ve savunma mekanizmalarını eş zamanlı olarak gözlemlemesini zorlaştırmaktadır. Bu nedenle kullanıcılar saldırının nasıl gerçekleştiğini

görebilmekte; ancak aynı sistemin güvenli sürümünde savunma mekanizmalarının davranışı nasıl değiştiğini doğrudan analiz edememektedir.

Bir diğer önemli eğitim platformu olan DVWA (Damn Vulnerable Web Application), web güvenliği eğitiminde uzun yıllardır kullanılan klasik laboratuvar ortamlarından biridir. DVWA özellikle başlangıç seviyesindeki güvenlik araştırmacıları ve öğrenciler için SQL Injection, Reflected XSS, Stored XSS, Command Injection, File Inclusion ve CSRF gibi temel web güvenlik açıklarının öğrenilmesini amaçlamaktadır. Platform PHP tabanlı geliştirilmiş olup kullanıcıların güvenlik açıklarını uygulamalı şekilde test edebilmesine olanak sağlamaktadır. Ayrıca güvenlik seviyelerinin “low”, “medium” ve “high” olarak değiştirilebilmesi sayesinde kullanıcılar aynı güvenlik açığının farklı koruma seviyelerindeki davranışını gözlemleyebilmektedir.

Ancak DVWA daha çok geleneksel web uygulama güvenliği yaklaşımına odaklanmaktadır. Platformun geliştirildiği dönem itibarıyla REST API mimarileri günümüzdeki kadar yaygın olmadığı için modern API güvenlik problemlerini kapsamlı şekilde ele almamaktadır. Özellikle JWT tabanlı kimlik doğrulama sistemleri, API rate limiting mekanizmaları, token manipülasyonu, Broken Object Level Authorization (BOLA), modern access control problemleri ve API gateway mantığı gibi güncel REST API güvenlik konuları platform içerisinde yeterince yer almamaktadır. Bu nedenle DVWA, temel web güvenliği eğitimi açısından güçlü olsa da modern REST API güvenliği konusunda sınırlı kalmaktadır.

PortSwigger tarafından geliştirilen Web Security Academy ise günümüzde en kapsamlı uygulamalı web güvenliği eğitim platformlarından biri olarak kabul edilmektedir. Platform; SQL Injection, XSS, CSRF, SSRF, authentication bypass, OAuth güvenliği, erişim kontrolü problemleri ve modern API güvenliği gibi çok geniş bir konu yelpazesini kapsamaktadır. Kullanıcılar platform üzerinde interaktif laboratuvarlar çözerek gerçek saldırı senaryolarını deneyimleyebilmektedir. Özellikle Burp Suite entegrasyonu sayesinde HTTP istekleri üzerinde detaylı analiz yapılabilmesi, platformun profesyonel güvenlik eğitiminde yaygın olarak kullanılmasını sağlamaktadır.

Web Security Academy’nin REST API güvenliği konusunda sunduğu laboratuvarlar modern sistemlere daha yakın senaryolar içermektedir. Özellikle JWT manipülasyonu, API authentication problemleri, erişim kontrolü zafiyetleri ve güvenlik doğrulama eksiklikleri gibi konular platform içerisinde uygulamalı olarak gösterilmektedir. Ancak platform daha çok saldırı perspektifine odaklanmakta; aynı saldırıların güvenli sistem mimarisinde nasıl engellendiğini doğrudan karşılaştırmalı biçimde göstermemektedir. Kullanıcılar güvenlik

açığının sömürebilmekte; ancak aynı sistemin güvenli versiyonunu inceleyerek savunma mekanizmalarının etkisini doğrudan analiz edememektedir.

Bu platformların ortak özelliklerinden biri, güvenlik açıklarının uygulamalı olarak öğrenilmesine önemli katkı sağlamalarıdır. Ancak mevcut sistemlerin büyük çoğunluğu ya yalnızca saldırı tarafına odaklanmakta ya da modern REST API güvenliğini tam kapsamlı biçimde ele almamaktadır. Ayrıca birçok platformda saldırı ve savunma mekanizmalarının aynı sistem üzerinde eş zamanlı karşılaştırılması mümkün değildir. Bu durum kullanıcıların güvenli backend geliştirme prensiplerini uygulamalı olarak anlamasını zorlaştırmaktadır.

Bu proje kapsamında geliştirilen laboratuvar sistemi ise mevcut platformlardan farklı olarak hem güvensiz hem de güvenli REST API ortamlarını aynı mimari üzerinde sunmayı hedeflemektedir. Böylece kullanıcılar aynı saldırının güvenlik önlemi olmayan bir sistemde nasıl başarılı olduğunu ve modern güvenlik mekanizmalarıyla güçlendirilmiş bir sistemde neden başarısız olduğunu doğrudan gözlemleyebilmektedir. Ayrıca proje özellikle REST API güvenliği üzerine odaklanarak JWT doğrulama, RBAC, WAF filtreleme, rate limiting ve modern API savunma mekanizmalarını uygulamalı biçimde göstermeyi amaçlamaktadır.

Sonuç olarak mevcut eğitim platformları siber güvenlik eğitimi açısından önemli katkılar sağlamakla birlikte; modern REST API güvenliği, saldırı-savunma karşılaştırması ve güvenli backend geliştirme farkındalığı konularında bazı eksikliklere sahiptir. Bu çalışma, söz konusu eksiklikleri azaltmayı ve REST API güvenliğini daha kapsamlı, karşılaştırmalı ve uygulamalı biçimde öğretmeyi hedefleyen bir laboratuvar ortamı sunmaktadır.

3.2. Akademik Çalışmalar

REST API güvenliği ve uygulamalı siber güvenlik eğitimi konusunda yapılan akademik çalışmalar, modern web servislerinin güvenlik açıklarına karşı oldukça hassas olduğunu ve özellikle uygulamalı eğitim yöntemlerinin güvenlik farkındalığını artırmada önemli rol oynadığını göstermektedir. Literatürde yer alan araştırmalar; REST API sistemlerinde erişim kontrolü, kimlik doğrulama mekanizmaları, veri güvenliği ve saldırı tespit süreçleri gibi konuların günümüzde kritik önem taşıdığını ortaya koymaktadır.

OWASP kapsamında tanımlanan API güvenlik açıklarının gerçek sistemlerde yaygın olarak bulunduğu birçok akademik çalışmada doğrulanmıştır. Vargas ve ark. (2020) tarafından gerçekleştirilen çalışmada, yüksek trafik hacmine sahip REST API servisleri analiz edilmiş ve incelenen sistemlerin %60'tan fazlasında yetersiz yetkilendirme (authorization)

kontrolleri bulunduđu tespit edilmiştir. Özellikle Broken Object Level Authorization (BOLA) zafiyetlerinin, kullanıcıların yetkisi olmayan verilere erişebilmesine neden olduğu belirtilmiştir. Çalışmada geliştiricilerin büyük bölümünün kimlik doğrulama mekanizmasını uygulamasına rağmen nesne seviyesinde erişim kontrolünü yeterince sağlamadığı vurgulanmıştır. Bu durumun özellikle kullanıcı verileri, sipariş sistemleri ve müşteri kayıtları gibi hassas veri içeren API servislerinde ciddi güvenlik riskleri oluşturduğu ifade edilmiştir.

Benzer şekilde API güvenliği üzerine yapılan diğer araştırmalar, modern REST servislerinde güvenlik kontrollerinin çoğu zaman işlevsellik odaklı geliştirme süreçlerinin gerisinde kaldığını göstermektedir. Özellikle hızlı geliştirme süreçleri, mikro servis mimarilerinin yaygınlaşması ve bulut tabanlı sistemlerin artışı; güvenlik yapılandırmalarında hatalara yol açabilmektedir. Literatürde yer alan birçok çalışmada SQL Injection, JWT manipölasyonu, güvenlik doğrulama eksiklikleri, yetersiz rate limiting mekanizmaları ve hatalı erişim kontrolü problemlerinin modern API sistemlerinde en sık karşılaşılan güvenlik problemleri arasında olduğu belirtilmektedir.

Akademik çalışmalar yalnızca güvenlik açıklarını incelemekle kalmamış, aynı zamanda güvenlik eğitimi yöntemlerinin etkinliği üzerine de yoğunlaşmıştır. Jensen ve ark. (2019) tarafından gerçekleştirilen araştırmada, uygulamalı CTF (Capture The Flag) tabanlı eğitim ortamlarının siber güvenlik farkındalığını artırmada geleneksel teorik öğretim yöntemlerine kıyasla çok daha etkili olduğu ortaya konmuştur. Araştırmada öğrenciler iki farklı gruba ayrılmış; bir gruba klasik teorik güvenlik eğitimi verilirken diğer grup uygulamalı laboratuvar ortamlarında saldırı senaryoları çözmüştür. Sonuçlar, uygulamalı eğitim alan öğrencilerin güvenlik açıklarını tanıma, saldırı mantığını anlama ve savunma mekanizmalarını yorumlama konusunda daha yüksek başarı gösterdiğini ortaya koymuştur.

Bu çalışmalar, özellikle uygulamalı laboratuvar ortamlarının öğrenme sürecindeki önemini vurgulamaktadır. Teorik eğitim modellerinde öğrenciler genellikle güvenlik açıklarının yalnızca tanımlarını öğrenirken; uygulamalı laboratuvar ortamlarında gerçek saldırı senaryolarını deneyimleyebilmekte ve sistem davranışlarını doğrudan gözlemleyebilmektedir. Bu yaklaşımın, güvenlik farkındalığının kalıcılığı açısından daha etkili olduğu akademik olarak desteklenmektedir.

Literatürde Docker tabanlı laboratuvar ortamları üzerine yapılan çalışmalar da dikkat çekmektedir. Özellikle konteyner tabanlı sistemlerin siber güvenlik eğitiminde sağladığı avantajlar birçok araştırmada vurgulanmıştır. Docker teknolojisi sayesinde laboratuvar ortamlarının farklı işletim sistemlerinde aynı şekilde çalıştırılabilmesi, eğitim süreçlerinde

standartlaşmayı kolaylaştırmaktadır. Ayrıca konteyner izolasyonu sayesinde güvenlik açıkları kontrollü biçimde çalıştırılabilmekte ve gerçek sistemlere zarar verme riski azaltılmaktadır.

Docker tabanlı izole laboratuvar ortamlarının, farklı donanım altyapılarında tekrar üretilebilir deney koşulları sağladığı ve bu özelliğin eğitim kalitesini önemli ölçüde artırdığı çeşitli akademik çalışmalarda belirtilmiştir. Özellikle tekrar üretilebilirlik (reproducibility) kavramı, siber güvenlik eğitiminde büyük önem taşımaktadır. Çünkü aynı laboratuvar ortamının farklı kullanıcılar tarafından aynı koşullarda çalıştırılabilmesi; eğitim süreçlerinin tutarlılığını ve güvenilirliğini artırmaktadır. Geleneksel sanal makine tabanlı laboratuvarlara kıyasla Docker konteynerlerinin daha hafif, hızlı ve taşınabilir olması da modern eğitim platformlarında yaygın şekilde tercih edilmesine neden olmaktadır.

Bunun yanında bazı akademik çalışmalar, güvenli ve güvensiz sistemlerin karşılaştırmalı olarak sunulmasının öğrenme sürecini olumlu etkilediğini göstermektedir. Kullanıcıların aynı saldırının hem savunmasız hem de korumalı sistem üzerindeki davranışını gözlemleyebilmesi, güvenlik mekanizmalarının çalışma mantığını daha iyi anlamalarını sağlamaktadır. Ancak literatürdeki mevcut platformların büyük çoğunluğu yalnızca saldırı odaklı laboratuvarlar sunmakta; güvenlik önlemlerinin sistem davranışını nasıl değiştirdiğini doğrudan göstermemektedir.

Bu çalışma, literatürde belirtilen eksiklikleri dikkate alarak tasarlanmıştır. Projede hem güvensiz hem de güvenli REST API ortamları aynı sistem içerisinde sunularak kullanıcıların saldırı ve savunma mekanizmalarını karşılaştırmalı biçimde analiz edebilmesi hedeflenmiştir. Ayrıca Docker tabanlı konteyner mimarisi kullanılarak tekrar üretilebilir, taşınabilir ve kontrollü bir laboratuvar ortamı oluşturulmuştur. Böylece proje, akademik çalışmalarda önerilen uygulamalı öğrenme yaklaşımını REST API güvenliği alanına uyarlayan eğitim odaklı bir sistem sunmaktadır.

Sonuç olarak literatürdeki çalışmalar; REST API güvenlik açıklarının modern sistemlerde oldukça yaygın olduğunu, uygulamalı laboratuvar ortamlarının güvenlik eğitimi açısından önemli avantajlar sağladığını ve Docker tabanlı tekrar üretilebilir sistemlerin eğitim kalitesini artırdığını göstermektedir. Bu proje ise söz konusu akademik bulgular doğrultusunda geliştirilmiş, REST API güvenliğini uygulamalı ve karşılaştırmalı biçimde öğretmeyi amaçlayan bir laboratuvar ortamı sunmaktadır.

3.3. API Fortress'in Farklılaştığı Noktalar

Bu çalışma kapsamında geliştirilen API Fortress sistemi, mevcut web güvenliği ve siber güvenlik eğitim platformlarından farklı olarak özellikle REST API güvenliği üzerine odaklanan, saldırı ve savunma mekanizmalarını aynı sistem içerisinde karşılaştırmalı biçimde sunan ve Docker tabanlı taşınabilir laboratuvar mimarisi kullanan uygulamalı bir eğitim platformu olarak tasarlanmıştır. Literatürde yer alan birçok platform güvenlik açıklarının sömürülmesine odaklanırken, API Fortress yalnızca saldırı mantığını öğretmeyi değil; aynı zamanda modern backend güvenlik mekanizmalarının sistem davranışını nasıl değiştirdiğini uygulamalı olarak göstermeyi hedeflemektedir.

API Fortress'in mevcut platformlardan ayrıldığı en önemli noktalardan biri, doğrudan REST API güvenliği üzerine yoğunlaşmasıdır. Günümüzde modern yazılım sistemlerinin büyük çoğunluğu frontend-backend ayrımına dayalı mimariler kullanmakta ve sistemler arasındaki veri iletişimi REST API servisleri üzerinden gerçekleştirilmektedir. Mobil uygulamalar, mikro servis mimarileri, bulut tabanlı sistemler ve modern web uygulamaları yoğun biçimde API teknolojilerine bağımlı hale gelmiştir. Buna rağmen mevcut eğitim platformlarının büyük bölümü hâlâ geleneksel web güvenliği açıklarına odaklanmakta; modern REST API zafiyetlerini kapsamlı şekilde ele almamaktadır.

API Fortress ise doğrudan OWASP API Security Top 10 kapsamındaki modern API güvenlik problemlerini hedef almaktadır. Sistem içerisinde Broken Object Level Authorization (BOLA), Broken Authentication, Security Misconfiguration, Injection saldırıları, yetersiz erişim kontrolü, token manipülasyonu, rate limiting eksiklikleri ve güvenli doğrulama problemleri gibi güncel REST API zafiyetleri uygulamalı olarak sunulmaktadır. Böylece kullanıcılar yalnızca klasik web uygulama güvenliğini değil; modern backend mimarilerinde karşılaşılan API güvenlik problemlerini de gerçekçi senaryolar üzerinden deneyimleyebilmektedir.

API Fortress'in farklılaştığı ikinci temel nokta, saldırı ve savunma ortamlarını aynı sistem içerisinde birlikte sunmasıdır. Mevcut güvenlik laboratuvarlarının büyük çoğunluğu yalnızca saldırı perspektifine odaklanmaktadır. Kullanıcılar güvenlik açıklarını sömürmeyi öğrenebilmekte; ancak aynı sistemin güvenli sürümünde savunma mekanizmalarının nasıl çalıştığını doğrudan gözlemleyememektedir. Bu durum, güvenli yazılım geliştirme süreçlerinin yeterince anlaşılmasını zorlaştırmaktadır.

API Fortress içerisinde ise aynı işlevleri yerine getiren iki farklı backend ortamı bulunmaktadır. İlk ortam kasıtlı olarak güvenlik açıkları içeren “vulnerable API” yapısını

temsil ederken; ikinci ortam modern güvenlik mekanizmalarıyla güçlendirilmiş “secure API” yapısını temsil etmektedir. Bu yaklaşım sayesinde kullanıcılar aynı saldırının iki farklı sistem üzerindeki davranışını karşılaştırmalı olarak inceleyebilmektedir.

Örneğin kullanıcılar güvensiz ortamda SQL Injection saldırısının başarılı şekilde çalıştığını gözlemleyebilirken; güvenli ortamda input validation, filtreleme mekanizmaları ve güvenli sorgu yapıları nedeniyle aynı saldırının neden başarısız olduğunu analiz edebilmektedir. Benzer şekilde JWT manipülasyonu, Broken Authentication veya yetkisiz veri erişimi gibi saldırıların güvenli backend mimarisinde nasıl engellendiği doğrudan incelenebilmektedir. Bu karşılaştırmalı yaklaşım, kullanıcıların yalnızca saldırı mantığını değil; güvenlik önlemlerinin sistem davranışı üzerindeki etkisini de anlamasını sağlamaktadır.

Sistem aynı zamanda modern savunma mekanizmalarını uygulamalı olarak göstermektedir. Güvenli API ortamında JWT tabanlı kimlik doğrulama sistemi, RBAC (Role-Based Access Control), regex tabanlı temel WAF filtreleme sistemi, rate limiting mekanizması, güvenli input validation işlemleri ve CORS güvenlik yapılandırmaları uygulanmıştır. Böylece kullanıcılar teorik güvenlik prensiplerini gerçek backend mimarisi üzerinde inceleyebilmekte ve modern güvenli yazılım geliştirme yaklaşımını uygulamalı biçimde deneyimleyebilmektedir.

API Fortress’in üçüncü önemli farklılığı ise Docker tabanlı taşınabilir laboratuvar mimarisidir. Geleneksel güvenlik laboratuvarlarında sistem kurulumu çoğu zaman karmaşık olmakta; işletim sistemi bağımlılıkları, paket sürüm uyumsuzlukları ve yapılandırma problemleri eğitim süreçlerini zorlaştırabilmektedir. Özellikle farklı donanım ve yazılım altyapılarında aynı laboratuvar ortamının tekrar oluşturulması önemli bir problem oluşturmaktadır.

API Fortress bu problemi Docker konteyner teknolojisi kullanarak çözmektedir. Sistem içerisindeki güvenli ve güvensiz backend servisleri konteyner tabanlı çalıştırılmakta ve tüm bağımlılıklar izole şekilde paketlenmektedir. Böylece kullanıcılar sistemi farklı işletim sistemlerinde veya geliştirme ortamlarında minimum kurulum işlemiyle çalıştırabilmektedir. Docker mimarisi aynı zamanda laboratuvar ortamının tekrar üretilebilirliğini artırmakta ve eğitim süreçlerinde standartlaşma sağlamaktadır.

Konteyner mimarisinin bir diğer avantajı ise güvenlik izolasyonudur. Güvensiz API servisleri kontrollü konteyner ortamlarında çalıştırıldığı için kullanıcıların saldırı testleri gerçek sistemlere zarar vermeden gerçekleştirilebilmektedir. Bu durum hem eğitim

güvenliğini artırmakta hem de saldırı senaryolarının kontrollü şekilde uygulanabilmesine olanak sağlamaktadır.

API Fortress ayrıca eğitim erişilebilirliği açısından da önemli avantajlar sunmaktadır. Sistem herhangi bir bulut ortamına, lokal geliştirme ortamına veya sanal laboratuvar altyapısına kolayca taşınabilmektedir. Böylece öğrenciler, araştırmacılar ve geliştiriciler farklı cihazlar üzerinde aynı laboratuvar ortamını çalıştırabilmekte ve standart bir eğitim deneyimi elde edebilmektedir.

Mevcut platformların önemli bir kısmı yalnızca challenge çözmeye odaklanırken API Fortress aynı zamanda backend mimarisinin anlaşılmasını da hedeflemektedir. Kullanıcılar yalnızca saldırıyı gerçekleştirmekle kalmamakta; API endpoint yapıları, JWT doğrulama süreçleri, rate limiting mantığı, WAF filtreleme mekanizmaları ve erişim kontrol sistemleri gibi backend bileşenlerinin çalışma mantığını da inceleyebilmektedir. Bu yönüyle sistem yalnızca bir saldırı laboratuvarı değil; aynı zamanda modern backend güvenliği eğitim platformu niteliği taşımaktadır.

Sonuç olarak API Fortress; doğrudan REST API güvenliği üzerine odaklanması, saldırı ve savunma ortamlarını karşılaştırmalı biçimde sunması, modern güvenlik mekanizmalarını uygulamalı olarak göstermesi ve Docker tabanlı taşınabilir laboratuvar mimarisi kullanması sayesinde mevcut eğitim platformlarından ayrılmaktadır. Bu özellikler sayesinde sistem, REST API güvenliği konusunda hem teknik hem de eğitimsel açıdan kapsamlı ve uygulama odaklı bir öğrenme ortamı sunmaktadır.

4. SİSTEM GEREKSİNİMLERİ

4.1. Fonksiyonel Gereksinimler

Sistemin fonksiyonel gereksinimleri aşağıdaki tabloda özetlenmektedir:

ID	Gereksinim	Açıklama
FR-1	Kullanıcı Yönetimi	Kullanıcılar sisteme kayıt olabilmeli, giriş yapabilmeli ve token tabanlı kimlik doğrulama gerçekleştirebilmelidir.
FR-2	Zafiyet Senaryoları	Güvensiz ortamda Mass Assignment, Broken Authentication, IDOR/BOLA ve BFLA zafiyetleri uygulanmış olmalıdır.
FR-3	CTF Hedefleri	Kullanıcılar endpoint'ler üzerinden saldırı gerçekleştirerek belirlenmiş hedeflere ulaşabilmelidir.
FR-4	Güvenli Ortam	Secure API ortamında tüm savunma mekanizmaları aktif ve işlevsel olmalıdır.
FR-5	Karşılaştırmalı Analiz	Kullanıcılar güvensiz ve güvenli ortamları aynı sistem üzerinde karşılaştırabilmelidir.
FR-6	Rol Yönetimi	Sistem; user, admin gibi farklı rolleri desteklemeli, rol bazlı erişim kontrolü uygulanmalıdır.

4.2. Güvenlik Gereksinimleri

Bu proje kapsamında geliştirilen API Fortress sistemi, modern REST API güvenlik prensiplerine uygun şekilde tasarlanmıştır. Sistem içerisinde hem saldırı senaryolarının uygulanabilmesi hem de güvenli backend mimarisinin gösterilebilmesi amacıyla çeşitli güvenlik gereksinimleri belirlenmiştir. Bu gereksinimler, güvenli API ortamında uygulanması zorunlu olan temel savunma mekanizmalarını tanımlamaktadır. Belirlenen güvenlik gereksinimleri; kimlik doğrulama, erişim kontrolü, saldırı filtreleme, parola güvenliği ve servis koruma mekanizmalarını kapsamaktadır.

NFR-1: Güvenli ortamda tüm endpoint'ler JWT doğrulaması gerektirmelidir.

Güvenli API ortamında yer alan tüm korumalı endpoint'lere erişim sağlanabilmesi için kullanıcıların geçerli bir JWT (JSON Web Token) ile doğrulanması gerekmektedir. JWT tabanlı kimlik doğrulama sistemi, modern REST API mimarilerinde yaygın olarak kullanılan stateless authentication yaklaşımını temsil etmektedir. Kullanıcı sisteme başarılı şekilde giriş yaptıktan sonra backend tarafından imzalanmış bir token üretilmekte ve kullanıcıya iletilmektedir. Sonraki API isteklerinde kullanıcı bu token'ı HTTP Authorization header'ı içerisinde göndermektedir.

Sistem, gelen token'ın doğruluğunu, imzasını, süresini ve kullanıcı bilgilerini kontrol ederek erişim izni vermektedir. Geçersiz, süresi dolmuş veya manipüle edilmiş token'lar sistem tarafından reddedilmektedir. Böylece yetkisiz kullanıcıların korumalı API endpoint'lerine erişmesi engellenmektedir. Bu yapı sayesinde modern backend sistemlerinde kullanılan güvenli kimlik doğrulama mantığı uygulamalı olarak gösterilmektedir.

Ayrıca JWT doğrulaması sayesinde backend tarafında oturum bilgisinin sunucu belleğinde tutulmasına gerek kalmamakta, böylece sistem daha ölçeklenebilir ve dağıtık mimariye uygun hale gelmektedir. Bu güvenlik gereksinimi özellikle kullanıcı verileri, yönetici işlemleri ve hassas endpoint'lerin korunması açısından kritik öneme sahiptir.

NFR-2: Rate limiting mekanizması, dakika başına istek sayısını sınırlandırmalıdır.

Sistem içerisinde brute force saldırıları, spam istekler ve servis yoğunlaştırma girişimlerini azaltmak amacıyla rate limiting mekanizması uygulanmıştır. Bu mekanizma, belirli bir istemcinin belirli süre içerisinde gönderebileceği maksimum istek sayısını sınırlandırmaktadır.

Örneğin bir kullanıcı kısa süre içerisinde sürekli login isteği gönderirse sistem bu davranışı şüpheli olarak değerlendirmekte ve belirlenen limit aşıldığında ilgili istemcinin yeni istekleri geçici olarak engellenmektedir. Böylece parola tahmin saldırıları ve otomatik saldırı araçlarının etkisi azaltılmaktadır.

Rate limiting sistemi özellikle aşağıdaki durumlarda kritik koruma sağlamaktadır:

- Brute force login saldırıları
- API spam istekleri
- Kaynak tüketimini artırmaya yönelik saldırılar
- Otomatik bot aktiviteleri
- Servis yoğunlaştırma girişimleri

Bu proje kapsamında Flask-Limiter mekanizması kullanılarak istemci IP adresi bazlı istek kontrolü uygulanmıştır. Böylece kullanıcıların belirlenen limitlerin üzerinde istek göndermesi engellenmiştir. Güvenli backend mimarilerinde rate limiting mekanizmaları servis sürekliliği ve sistem kararlılığı açısından önemli güvenlik katmanlarından biridir.

NFR-3: WAF sistemi SQL Injection ve XSS içeren istekleri engelleyebilmelidir.

Güvenli API ortamında temel seviyede bir WAF (Web Application Firewall) sistemi uygulanmıştır. Bu sistemin amacı, kullanıcıdan gelen zararlı girdileri analiz ederek potansiyel saldırı girişimlerini engellemektir.

Sistem özellikle aşağıdaki saldırı türlerini tespit etmeye odaklanmaktadır:

- SQL Injection
- Cross-Site Scripting (XSS)
- Zararlı script girişimleri
- Şüpheli SQL sorgu yapıları
- Veri manipülasyonu girişimleri

WAF sistemi, gelen HTTP request içeriklerini regex tabanlı filtreleme mekanizmalarıyla incelemektedir. Örneğin:

```
OR 1=1UNION SELECTDROP TABLE<script>
```

gibi zararlı ifadeler tespit edildiğinde sistem isteği engellemekte ve kullanıcıya hata cevabı döndürmektedir.

Bu yaklaşım gerçek kurumsal WAF sistemlerinin basitleştirilmiş bir simülasyonu niteliğindedir. Amaç yalnızca saldırıyı engellemek değil; aynı zamanda kullanıcıların modern backend güvenlik filtreleme mantığını uygulamalı olarak anlayabilmesini sağlamaktır.

WAF mekanizması özellikle input validation süreçlerinin önemini göstermektedir. Güvensiz backend sistemlerinde filtrelenmeyen kullanıcı girdileri ciddi güvenlik açıklarına neden olabilirken; güvenli sistemlerde giriş verilerinin kontrol edilmesi saldırı yüzeyini önemli ölçüde azaltmaktadır.

NFR-4: Kullanıcı parolaları düz metin olarak saklanmamalı; bcrypt ile hashlenerek depolanmalıdır.

Sistem güvenliği açısından kullanıcı parolalarının düz metin (plain text) olarak saklanması kesinlikle yasaktır. Güvenli API ortamında kullanıcı şifreleri bcrypt algoritması kullanılarak hashlenmekte ve yalnızca hash değerleri veritabanında saklanmaktadır.

Bu yaklaşımın temel amacı, veritabanı sızıntısı durumunda kullanıcı şifrelerinin doğrudan ele geçirilmesini engellemektir. Eğer şifreler düz metin olarak saklanırsa veritabanına erişen saldırganlar tüm kullanıcı hesaplarını kolayca ele geçirebilir. Ancak bcrypt ile hashlenmiş parolalar geri döndürülemez yapıda olduğu için saldırganlar gerçek şifreleri doğrudan elde edememektedir.

4.2.1. Bcrypt algoritması aynı zamanda:

- Salt mekanizması kullanır,
- Rainbow table saldırılarını zorlaştırır,
- Brute force saldırı maliyetini artırır,

- Her parola için farklı hash üretir.

Kullanıcı giriş işlemi sırasında sistem, girilen şifreyi yeniden hashleyerek veritabanındaki hash değeriyle karşılaştırmaktadır. Böylece gerçek parola hiçbir zaman doğrudan saklanmamakta veya sistem içerisinde açık biçimde tutulmamaktadır.

Bu güvenlik gereksinimi modern kimlik doğrulama sistemlerinin temel prensiplerinden biridir ve güvenli backend geliştirme süreçlerinde kritik öneme sahiptir.

NFR-5: Rol tabanlı yetkilendirme, kullanıcı ve yönetici işlevlerini birbirinden kesin biçimde ayırmalıdır.

Sistem içerisinde Role-Based Access Control (RBAC) yaklaşımı uygulanmıştır. Bu yapı sayesinde kullanıcıların sistem içerisindeki yetkileri roller üzerinden kontrol edilmektedir.

4.2.2. Projede temel olarak iki farklı rol bulunmaktadır:

- Normal kullanıcı (user)
- Yönetici (admin)

Her rol yalnızca kendisine izin verilen işlemleri gerçekleştirebilmektedir. Örneğin normal kullanıcılar yalnızca kendi verilerine erişebilirken; yönetici kullanıcılar sistem yönetimi, kullanıcı listeleme veya kritik işlemleri gerçekleştirebilmektedir.

4.2.3. RBAC sisteminin temel amacı:

- Yetkisiz erişimleri engellemek,
- Ayrıcalık kontrolü sağlamak,
- Hassas işlemleri korumak,
- Kullanıcı izolasyonu oluşturmaktır.

Sistem, her API isteğinde kullanıcının token bilgilerini analiz ederek ilgili rolü kontrol etmektedir. Eğer kullanıcı gerekli yetkiye sahip değilse sistem erişimi reddetmektedir.

Böylece kullanıcıların yönetici işlemlerine erişmesi veya başka kullanıcıların verilerini görüntülemesi engellenmektedir.

Bu güvenlik gereksinimi özellikle Broken Access Control ve Broken Object Level Authorization (BOLA) gibi modern API zafiyetlerini önlemek açısından büyük önem taşımaktadır. Çünkü birçok gerçek dünya saldırısı yetersiz erişim kontrolü mekanizmalarından kaynaklanmaktadır.

Sonuç olarak bu güvenlik gereksinimleri, API Fortress sisteminin modern REST API güvenlik prensiplerine uygun biçimde çalışmasını sağlamak amacıyla belirlenmiştir. JWT doğrulama, rate limiting, WAF filtreleme, bcrypt tabanlı parola güvenliği ve rol tabanlı erişim kontrolü gibi mekanizmalar sayesinde sistem yalnızca saldırı senaryolarını göstermekle kalmamakta; aynı zamanda güvenli backend geliştirme yaklaşımını da uygulamalı olarak sergilemektedir.

4.3. Teknik Gereksinimler

API Fortress sisteminin geliştirme sürecinde modern backend mimarisi, sürdürülebilir yazılım geliştirme yaklaşımı ve güvenli sistem tasarımı dikkate alınarak çeşitli teknik gereksinimler belirlenmiştir. Bu gereksinimler; sistemin geliştirme altyapısını, backend mimarisini, veritabanı yönetimini, konteyner yapısını ve güvenli yapılandırma süreçlerini tanımlamaktadır. Belirlenen teknik gereksinimler aynı zamanda projenin modüler, taşınabilir, sürdürülebilir ve gerçek dünya backend sistemlerine yakın bir yapıda geliştirilmesini amaçlamaktadır.

TR-1: Backend geliştirme için Python 3.9+ ve Flask framework kullanılmalıdır.

Sistemin backend katmanı Python programlama dili kullanılarak geliştirilmiştir. Python'un tercih edilmesinin temel nedenleri arasında hızlı geliştirme imkânı sunması, geniş kütüphane desteğine sahip olması, okunabilir sözdizimi ve modern web teknolojileriyle uyumlu yapısı yer almaktadır. Özellikle siber güvenlik, veri işleme ve backend geliştirme alanlarında Python yaygın olarak kullanılmaktadır.

Projede minimum Python 3.9 sürümü hedeflenmiştir. Bunun nedeni modern Python sürümleriyle birlikte gelen performans iyileştirmeleri, gelişmiş type hint desteği, güvenlik güncellemeleri ve güncel kütüphanelerle uyumluluktur. Ayrıca Docker ortamında çalışan backend servislerinin stabil biçimde yönetilebilmesi açısından güncel Python sürümleri tercih edilmiştir.

Backend framework olarak Flask kullanılmıştır. Flask, hafif (lightweight) ve esnek yapısı sayesinde REST API geliştirme süreçleri için oldukça uygun bir framework'tür. Flask'ın tercih edilme nedenleri şunlardır:

- Minimal ve modüler yapı sunması
- REST API geliştirmeye uygun olması
- Blueprint mimarisini desteklemesi
- Kolay genişletilebilir olması
- Güvenlik ve middleware entegrasyonlarının kolay yapılabilmesi
- Eğitim ve laboratuvar ortamları için anlaşılır yapı sağlaması

Ayrıca Flask ekosistemi sayesinde JWT doğrulama, rate limiting, ORM yönetimi ve CORS entegrasyonu gibi güvenlik bileşenleri sisteme kolayca entegre edilebilmiştir. Böylece proje hem modern backend geliştirme yaklaşımını hem de güvenlik odaklı API mimarisini uygulamalı olarak gösterebilecek bir altyapıya kavuşmuştur.

TR-2: Veritabanı işlemleri SQLAlchemy ORM üzerinden yürütülmelidir.

Sistem içerisinde veritabanı işlemleri doğrudan ham SQL sorguları ile değil, SQLAlchemy ORM (Object Relational Mapping) katmanı üzerinden yürütülmektedir. ORM yaklaşımının temel amacı veritabanı işlemlerini nesne yönelimli programlama mantığıyla yönetebilmektir.

SQLAlchemy sayesinde veritabanındaki tablolar Python sınıflarıyla temsil edilmektedir. Örneğin kullanıcı tablosu bir User modeli üzerinden tanımlanmakta ve kayıt işlemleri Python nesneleri üzerinden gerçekleştirilmektedir. Böylece geliştiriciler doğrudan karmaşık SQL sorguları yazmak yerine daha okunabilir ve sürdürülebilir kod yapıları oluşturabilmektedir.

4.3.1. ORM kullanımının sağladığı temel avantajlar şunlardır:

- Kod okunabilirliğinin artması
- Veritabanı bağımsızlığı sağlanması
- SQL sorgularının merkezi yönetilebilmesi

- Model tabanlı veri yönetimi
- Daha güvenli sorgu yapıları oluşturulması
- Geliştirme sürecinin hızlanması

Ayrıca SQLAlchemy parameterized query yaklaşımı kullandığı için SQL Injection riskinin azaltılmasına katkı sağlamaktadır. Güvensiz API ortamında bazı endpoint'lerde bilinçli olarak zafiyet oluşturulsa da güvenli backend ortamında ORM yapısı güvenli veri işleme mantığını temsil etmektedir.

Projede SQLAlchemy kullanılması aynı zamanda gerçek dünya backend mimarilerine daha yakın bir yapı oluşturulmasını sağlamaktadır. Günümüzde birçok profesyonel Python backend sistemi ORM tabanlı veri yönetimi kullanmaktadır.

TR-3: Sistem Docker konteyner ortamında çalıştırılabilir olmalıdır.

API Fortress sistemi Docker konteyner mimarisi kullanılarak geliştirilmiştir. Docker teknolojisinin kullanılmasının temel amacı sistemin taşınabilir, izole ve tekrar üretilebilir bir laboratuvar ortamı sunmasını sağlamaktır.

Geleneksel geliştirme süreçlerinde farklı işletim sistemleri, bağımlılık sürümleri ve ortam yapılandırmaları çeşitli uyumsuzluk problemlerine neden olabilmektedir. Docker konteyner teknolojisi sayesinde uygulamanın ihtiyaç duyduğu tüm bağımlılıklar aynı konteyner içerisinde paketlenmekte ve sistemin farklı ortamlarda aynı şekilde çalışması sağlanmaktadır.

Docker kullanımının projeye sağladığı avantajlar şunlardır:

- Platform bağımsız çalışabilme
- Hızlı kurulum ve dağıtım
- İzole çalışma ortamı
- Tekrar üretilebilir laboratuvar yapısı
- Güvensiz servislerin kontrollü şekilde çalıştırılması
- Geliştirme ortamı standartlaşması
- Mikro servis mimarisine uygun yapı

Projede hem güvenli hem de güvensiz backend servisleri ayrı konteynerler içerisinde çalıştırılabilmektedir. Böylece kullanıcılar saldırı testlerini kontrollü laboratuvar ortamında gerçekleştirebilmekte ve sistemin gerçek cihazlara zarar verme riski azaltılmaktadır.

Docker kullanımı aynı zamanda eğitim erişilebilirliğini de artırmaktadır. Kullanıcılar sistemi yalnızca birkaç komut ile kendi bilgisayarlarında çalıştırabilmekte ve karmaşık kurulum süreçleriyle uğraşmadan laboratuvar ortamına erişebilmektedir.

TR-4: API endpoint'leri Flask Blueprint mimarisiyle modüler olarak organize edilmelidir.

Sistemin backend mimarisinde Flask Blueprint yapısı kullanılmıştır. Blueprint mimarisinin temel amacı büyük ölçekli backend sistemlerini modüler hale getirmek ve farklı işlevleri birbirinden ayrılmış dosyalar halinde organize edebilmektir.

Projede kullanıcı işlemleri, kimlik doğrulama işlemleri ve veri yönetimi gibi farklı API servisleri ayrı route dosyalarına bölünmüştür. Örneğin:

```
routes_auth.py routes_users.py routes_items.py
```

gibi modüller birbirinden bağımsız şekilde yapılandırılmıştır.

Blueprint mimarisinin kullanılmasının temel avantajları şunlardır:

- Kod karmaşıklığını azaltmak
- Büyük projelerde okunabilirliği artırmak
- Endpoint yönetimini kolaylaştırmak
- Modüller arası bağımsızlık sağlamak
- Yeni servislerin kolay eklenebilmesi
- Test süreçlerini kolaylaştırmak
- Bakım süreçlerini hızlandırmak

Bu yapı sayesinde backend sistemi tek dosyada çalışan karmaşık bir yapı olmaktan çıkılarak profesyonel backend mimarilerine benzer şekilde organize edilmiştir. Ayrıca modüler yapı sayesinde güvenli ve güvensiz backend ortamları daha kolay yönetilebilmektedir.

Blueprint mimarisi aynı zamanda REST API geliştirme süreçlerinde ölçeklenebilirlik açısından önemli avantaj sağlamaktadır. Yeni endpoint'lerin veya güvenlik mekanizmalarının sisteme eklenmesi mevcut yapıyı bozmadan gerçekleştirilebilmektedir.

TR-5: Güvenli ortamda gizli bilgiler (secret key, DB şifresi) environment variable olarak yönetilmelidir.

Sistem güvenliği açısından kritik yapılandırma bilgilerinin doğrudan kaynak kod içerisine yazılması büyük güvenlik riski oluşturmaktadır. Bu nedenle güvenli backend ortamında gizli bilgiler environment variable mekanizması kullanılarak yönetilmektedir.

Environment variable yapısı sayesinde aşağıdaki bilgiler kod içerisine gömülmeden yönetilebilmektedir:

- JWT secret key
- Veritabanı kullanıcı adı ve şifresi
- Flask secret key
- Debug ayarları
- API güvenlik anahtarları
- Konteyner yapılandırma bilgileri

4.3.2. Bu yaklaşımın temel avantajları şunlardır:

- Hassas bilgilerin kaynak koddan ayrılması
- GitHub gibi platformlarda secret sızıntılarının önlenmesi
- Farklı ortamlar için ayrı yapılandırma sağlanması
- Production ve development ortamlarının ayrıştırılması
- Güvenli dağıtım süreçlerinin desteklenmesi

Projede .env dosyaları kullanılarak ortam değişkenleri yönetilmiştir. Flask uygulaması başlatılırken bu değişkenler yüklenmekte ve backend servisleri gerekli yapılandırmaları güvenli şekilde kullanabilmektedir.

Bu gereksinim modern backend geliştirme süreçlerinde kritik güvenlik standartlarından biridir. Özellikle gerçek dünya sistemlerinde gizli bilgilerin kaynak kod içerisine yazılması ciddi güvenlik problemlerine neden olabilmektedir. API Fortress içerisinde bu yaklaşım uygulanarak güvenli yapılandırma yönetimi prensibi uygulamalı biçimde gösterilmiştir.

Sonuç olarak belirlenen teknik gereksinimler; sistemin modern backend geliştirme prensiplerine uygun, modüler, güvenli, taşınabilir ve sürdürülebilir bir yapıda geliştirilmesini sağlamaktadır. Python ve Flask tabanlı backend mimarisi, SQLAlchemy ORM kullanımı, Docker konteyner yapısı, Blueprint tabanlı modüler organizasyon ve environment variable yönetimi sayesinde API Fortress hem eğitimsel hem de teknik açıdan gerçek dünya backend sistemlerine yakın bir laboratuvar ortamı sunmaktadır.

5. SİSTEM TASARIMI VE YAZILIM MİMARİSİ

5.1. Genel Mimari

API Fortress sistemi, modern REST API güvenlik mekanizmalarının uygulamalı olarak analiz edilebilmesi amacıyla geliştirilmiş modüler bir saldırı ve savunma laboratuvarı olarak tasarlanmıştır. Sistem mimarisi oluşturulurken ölçeklenebilirlik, modülerlik, güvenlik, taşınabilirlik ve sürdürülebilirlik prensipleri dikkate alınmıştır. Proje yalnızca güvenlik açıklarını gösteren basit bir demo uygulaması olarak değil; gerçek dünya backend mimarilerine benzer şekilde çalışan eğitim odaklı bir laboratuvar sistemi olarak geliştirilmiştir.

5.1.1. Sistem temel olarak üç ana katmandan oluşmaktadır:

- Backend katmanı
- Güvenlik katmanı
- Veritabanı katmanı

Bu katmanlar Docker konteyner mimarisi içerisinde izole biçimde çalıştırılmakta ve birbirleriyle REST API mantığı üzerinden iletişim kurmaktadır. Böylece sistem hem modüler bir yapıya sahip olmakta hem de farklı geliştirme ortamlarında kolayca çalıştırılabilmektedir.

Sistemin backend geliştirme sürecinde Python programlama dili ve Flask framework'ü kullanılmıştır. Flask'ın hafif ve modüler yapısı sayesinde REST API endpoint'leri kolayca organize edilebilmiş, güvenlik bileşenleri sisteme entegre edilebilmiş ve laboratuvar ortamı daha anlaşılır hale getirilmiştir. Ayrıca Flask Blueprint mimarisi kullanılarak auth, users ve items gibi farklı servisler ayrı modüller halinde yapılandırılmıştır.

Sistemin genel katmanlı mimarisi aşağıdaki yapıdan oluşmaktadır:

Bu mimari yapı sayesinde istemci tarafı ile veritabanı arasında doğrudan bağlantı kurulmadan, tüm işlemler kontrollü backend katmanı üzerinden gerçekleştirilmektedir. Böylece veri güvenliği sağlanmakta ve sistem üzerinde merkezi kontrol mekanizması oluşturulmaktadır.

5.1.2. İstemci Katmanı

Sistemin en üst katmanında istemci tarafı yer almaktadır. İstemci; kullanıcıların REST API endpoint'lerine HTTP istekleri gönderebildiği bölümü temsil etmektedir. Bu istemciler bir web arayüzü, Postman, tarayıcı tabanlı REST istemcileri veya komut satırı araçları olabilir. Kullanıcılar sistemdeki saldırı ve savunma senaryolarını bu istemci katmanı üzerinden test etmektedir.

İstemci tarafında gerçekleştirilen temel işlemler şunlardır:

- Kullanıcı kayıt işlemleri
- Kullanıcı giriş işlemleri
- JWT token kullanımı
- Veri listeleme
- Veri oluşturma
- Veri güncelleme
- Veri silme
- Güvenlik açıklarının test edilmesi
- Yetkilendirme kontrollerinin analiz edilmesi

İstemci ile sunucu arasındaki veri iletişimi tamamen HTTP protokolü üzerinden gerçekleştirilmektedir. İstekler REST API mantığına uygun olarak belirli endpoint'lere gönderilmekte ve sunucu tarafından işlenmektedir.

5.1.3. Flask API Katmanı

Sistemin merkezinde Flask tabanlı REST API katmanı bulunmaktadır. Bu katman, istemcilerden gelen HTTP isteklerini işleyen ana backend bileşenidir. API katmanı aynı zamanda sistemin iş mantığını (business logic) yönetmekte ve güvenlik kontrollerini uygulamaktadır.

5.1.4. Flask API katmanının temel görevleri şunlardır:

- HTTP isteklerini almak
- Endpoint yönlendirmelerini gerçekleştirmek
- Kullanıcı doğrulama işlemleri yapmak
- JWT token kontrollerini yürütmek
- Yetkilendirme süreçlerini yönetmek
- Veritabanı işlemleri gerçekleştirmek
- Güvenlik filtrelerini uygulamak
- JSON formatında yanıt döndürmek

Sistem içerisinde Flask Blueprint mimarisi kullanılmıştır. Bu yapı sayesinde API endpoint'leri işlevlerine göre modüllere ayrılmıştır:

```
auth -> Kimlik dogrulama islemleri  
users -> Kullanici islemleri  
items -> Veri ve saldiri senaryolari
```

Bu modüler yapı sayesinde sistem daha okunabilir, sürdürülebilir ve geliştirilebilir hale gelmiştir. Ayrıca güvenli ve güvensiz backend servislerinin yönetimi de kolaylaştırılmıştır.

5.1.5. Güvenlik Katmanı

API Fortress sisteminin en önemli bileşenlerinden biri güvenlik katmanıdır. Bu katman modern REST API sistemlerinde kullanılan temel savunma mekanizmalarını uygulamalı olarak göstermektedir.

Güvenlik katmanı içerisinde şu bileşenler bulunmaktadır:

5.1.6. JWT Authentication

JWT tabanlı kimlik doğrulama sistemi kullanılarak kullanıcıların korumalı endpoint'lere erişmeden önce doğrulanması sağlanmaktadır. Kullanıcı sisteme giriş yaptıktan sonra backend tarafından imzalanmış bir token üretilmekte ve sonraki isteklerde bu token doğrulanmaktadır.

5.1.7. Bu yapı sayesinde:

- Yetkisiz erişimler engellenmekte,
- Stateless authentication sağlanmakta,
- Modern REST API güvenlik mantığı uygulanmaktadır.

5.1.8. RBAC (Role-Based Access Control)

Rol tabanlı erişim kontrol sistemi sayesinde kullanıcı ve yönetici yetkileri birbirinden ayrılmıştır.

5.1.9. Örneğin:

- Normal kullanıcılar yalnızca kendi işlemlerini gerçekleştirebilir,
- Yönetici kullanıcılar ise sistem yönetim işlemlerine erişebilir.

Bu yapı modern API sistemlerinde sık karşılaşılan Broken Access Control problemlerini önlemeyi amaçlamaktadır.

5.1.10. Rate Limiting

Rate limiting mekanizması kullanıcıların belirli süre içerisinde gönderebileceği maksimum istek sayısını sınırlandırmaktadır.

Bu yapı sayesinde: Brute force saldırıları, Spam istekler, Kaynak tüketim saldırıları, Otomatik bot aktiviteleri engellenmeye çalışılmaktadır.

5.1.11. WAF (Web Application Firewall)

Regex tabanlı temel WAF sistemi kullanılarak zararlı payload'lar filtrelenmektedir.

5.1.12. Örneğin:

- SQL Injection denemeleri
- XSS payload'ları
- Zararlı script içerikleri

tespit edildiğinde sistem isteği reddetmektedir.

Bu yapı güvenli backend sistemlerinde input validation süreçlerinin önemini göstermektedir.

5.1.13. Veritabanı Katmanı

Sistemin veri yönetim süreçleri SQLAlchemy ORM katmanı üzerinden gerçekleştirilmektedir. Veritabanı katmanının temel amacı kullanıcı bilgilerini, kimlik doğrulama verilerini ve laboratuvar işlemlerini güvenli şekilde depolamaktır.

Projede SQLite veritabanı kullanılmıştır. SQLite'in tercih edilme nedenleri şunlardır:

- Hafif yapı sunması
- Kurulum gerektirmemesi
- Eğitim ve laboratuvar ortamları için uygun olması
- Docker içerisinde kolay kullanılabilmesi
- Geliştirme süreçlerini hızlandırması

Veritabanı işlemleri SQLAlchemy ORM üzerinden yürütülmektedir. Böylece veritabanı tabloları Python sınıflarıyla temsil edilmekte ve veri işlemleri nesne yönelimli programlama mantığıyla gerçekleştirilmektedir.

5.1.14. Örneğin:

- User modeli kullanıcı tablosunu,
- Item modeli veri nesnelerini temsil etmektedir.

5.1.15. Bu yaklaşım:

- Kod okunabilirliğini artırmakta,
- Güvenli sorgu yapıları oluşturmakta,
- Veritabanı yönetimini kolaylaştırmaktadır.

5.1.16. REST API İletişim Yapısı

Sistem içerisinde istemci ve sunucu arasındaki veri iletişimi REST API mimarisi üzerinden gerçekleştirilmektedir. İstemciler HTTP istekleri aracılığıyla endpoint'lere erişmekte, sunucu tarafında gerçekleştirilen işlemler sonucunda JSON formatında yanıt döndürülmektedir.

Projede kullanılan temel HTTP metodları şunlardır:

HTTP Metodu	Görev
GET	Veri listeleme veya görüntüleme
POST	Yeni veri oluşturma
PUT	Var olan veriyi güncelleme
DELETE	Veri silme

Örneğin kullanıcı giriş işlemi şu akış üzerinden gerçekleşmektedir:

İstemci → POST /login → Flask API → JWT oluşturma → JSON Response → İstemci

Benzer şekilde korumalı endpoint'lere erişim sırasında sistem JWT doğrulaması gerçekleştirmekte ve kullanıcı yetkilerini kontrol etmektedir.

5.1.17. Docker Tabanlı Çalışma Ortamı

API Fortress sistemi Docker konteyner mimarisi kullanılarak çalıştırılmaktadır. Docker kullanımı sayesinde sistemin tüm bağımlılıkları izole şekilde paketlenilebilmekte ve farklı geliştirme ortamlarında aynı yapı korunabilmektedir.

5.1.18. Docker mimarisi sayesinde:

- Kurulum süreçleri kolaylaşmakta,
- Ortam bağımlılıkları azaltılmakta,
- Laboratuvar tekrar üretilebilir hale gelmekte,
- Güvensiz servisler kontrollü şekilde çalıştırılabilmektedir.

Bu yaklaşım aynı zamanda modern mikro servis mimarilerine benzer bir çalışma mantığı sunmaktadır.

Sonuç olarak API Fortress'in genel mimarisi; modüler Flask backend yapısı, REST API iletişim modeli, güvenlik katmanı, ORM tabanlı veritabanı yönetimi ve Docker konteyner mimarisi üzerine kurulmuştur. Bu yapı sayesinde sistem hem modern backend geliştirme prensiplerini hem de REST API güvenlik mekanizmalarını uygulamalı biçimde gösteren kapsamlı bir saldırı ve savunma laboratuvarı ortamı sunmaktadır.

5.2. Modüler Backend Yapısı

API Fortress projesi, güvenli ve güvensiz REST API ortamlarını birbirinden ayıracak şekilde modüler bir dosya yapısı ile organize edilmiştir. Bu yapı sayesinde proje hem okunabilir hem de sürdürülebilir hale getirilmiştir. Sistem içerisinde iki farklı backend ortamı bulunduğu için dosya organizasyonu özellikle karşılaştırmalı analiz yapılabilecek şekilde tasarlanmıştır.

Projenin temel dosya yapısı aşağıdaki gibidir:

```
api_fortress/  
|-- insecure_api/  
|   |-- app.py
```

```
|    |-- routes_auth.py
|    |-- routes_users.py
|    `-- routes_items.py
|-- secure_api/
|    |-- app.py
|    |-- routes_auth.py
|    |-- routes_users.py
|    `-- routes_items.py
|-- common/
|    |-- config.py
|    |-- db.py
|    `-- models.py
|-- docker-compose.yml
`-- requirements.txt
```

Bu dosya yapısında proje üç ana mantıksal bölüme ayrılmıştır: `insecure_api`, `secure_api` ve `common`. Bu ayırım, sistemin saldırı ve savunma taraflarını net biçimde ayırmak için tercih edilmiştir. Böylece kullanıcılar aynı API mantığının güvenli ve güvensiz sürümlerini dosya düzeyinde de karşılaştırabilir.

5.2.1. `insecure_api` Klasörü

`insecure_api` klasörü, kasıtlı olarak güvenlik açıkları içeren REST API ortamını temsil etmektedir. Bu klasördeki backend, eğitim ve test amacıyla savunmasız şekilde tasarlanmıştır. Amaç, kullanıcıların gerçek sistemlerde karşılaşılabilecek API güvenlik açıklarını kontrollü bir ortamda gözlemleyebilmesini sağlamaktır.

Bu ortamda bulunan endpoint'ler temel API işlevlerini yerine getirir; ancak güvenlik kontrolleri bilinçli olarak eksik veya zayıf bırakılmıştır. Örneğin kimlik doğrulama kontrolleri yetersiz olabilir, kullanıcı rolleri yeterince denetlenmeyebilir, input validation işlemleri eksik olabilir veya bazı endpoint'ler zararlı girdilere karşı savunmasız bırakılabilir. Böylece SQL Injection, yetkisiz erişim, zayıf authentication, eksik authorization ve hatalı veri işleme gibi senaryolar test edilebilir.

`insecure_api/app.py` dosyası, güvensiz Flask uygulamasının başlangıç noktasıdır. Bu dosyada Flask uygulaması başlatılır, gerekli route dosyaları uygulamaya bağlanır ve API servisi

çalışır hale getirilir. Güvensiz ortamda amaç savunma mekanizmalarını minimumda tutarak saldırı senaryolarının etkisini gösterebilmektir.

`insecure_api/routes_auth.py` dosyası kayıt ve giriş endpoint'lerini içerir. Bu bölümde kullanıcıların sisteme kayıt olması ve giriş yapması sağlanır. Ancak bu ortamda authentication mekanizmaları güvenli ortama göre daha zayıf yapılandırılmıştır. Bu durum, kullanıcıların zayıf kimlik doğrulama süreçlerinin nasıl güvenlik açığı oluşturduğunu incelemesine imkân tanır.

`insecure_api/routes_users.py` dosyası kullanıcı yönetimi işlemlerini içerir. Kullanıcı listeleme, kullanıcı detaylarını görüntüleme veya kullanıcı bilgilerini değiştirme gibi işlemler bu dosya üzerinden yürütülür. Güvensiz ortamda bu endpoint'ler üzerinde yeterli rol ve yetki kontrolü bulunmadığında, kullanıcıların yetkisiz verilere erişebilmesi gibi zafiyetler gözlemlenebilir.

`insecure_api/routes_items.py` dosyası ise temel CRUD işlemlerinin bulunduğu bölümdür. Veri oluşturma, listeleme, güncelleme ve silme işlemleri bu modül üzerinden gerçekleştirilir. Bu dosya aynı zamanda injection, eksik doğrulama ve hatalı veri işleme gibi saldırı senaryolarının gösterilebildiği önemli bir alandır.

5.2.2. `secure_api` Klasörü

`secure_api` klasörü, modern güvenlik mekanizmalarıyla güçlendirilmiş REST API ortamını temsil etmektedir. Bu bölümde, güvensiz ortamda yer alan temel işlevler korunmuş; ancak bu işlevlerin üzerine güvenlik katmanları eklenmiştir. Böylece kullanıcılar aynı işlemlerin güvenli mimaride nasıl yürütüldüğünü inceleyebilmektedir.

`secure_api/app.py` dosyası güvenli Flask uygulamasının başlangıç noktasıdır. Bu dosyada Flask uygulaması başlatılırken güvenlik bileşenleri de sisteme entegre edilir. CORS ayarları, rate limiting mekanizması, WAF kontrolleri, JWT doğrulama süreçleri ve Blueprint bağlantıları bu yapı içerisinde yönetilir. Güvenli uygulama, istemcilerden gelen istekleri doğrudan işlemek yerine önce güvenlik kontrollerinden geçirir.

`secure_api/routes_auth.py` dosyası JWT destekli authentication işlemlerini içerir. Kullanıcı başarılı şekilde giriş yaptığındaki backend tarafından imzalanmış bir JWT token üretilir. Bu token, kullanıcının sonraki isteklerde kimliğini doğrulamak için kullanılır. Ayrıca güvenli ortamda kullanıcı parolalarının düz metin olarak saklanmaması ve bcrypt ile hashlenmesi gibi güvenlik önlemleri uygulanır.

secure_api/routes_users.py dosyası RBAC korumalı kullanıcı işlemlerini içerir. Bu bölümde kullanıcı ve yönetici rolleri birbirinden ayrılır. Normal kullanıcılar yalnızca kendilerine izin verilen işlemleri gerçekleştirebilirken, admin kullanıcılar daha geniş yönetim yetkilerine sahip olabilir. Böylece Broken Access Control ve yetkisiz erişim riskleri azaltılır.

secure_api/routes_items.py dosyası WAF ve rate limit korumalı CRUD işlemlerini içerir. Bu modül üzerinden gelen veri işleme istekleri, güvenlik filtrelerinden geçirilir. Zararlı SQL Injection veya XSS payload'ları WAF tarafından engellenebilir. Aynı zamanda rate limiting mekanizması, kısa sürede çok fazla istek gönderilmesini sınırlandırarak brute force ve spam saldırılarına karşı koruma sağlar.

5.2.3. common Klasörü

common klasörü, hem güvenli hem de güvensiz API ortamları tarafından ortak kullanılan yapıların bulunduğu bölümdür. Bu klasörün amacı kod tekrarını azaltmak ve temel sistem bileşenlerini merkezi bir yerde toplamaktır.

common/config.py dosyası sistem yapılandırmalarını içerir. Veritabanı bağlantı adresi, secret key, debug ayarları ve environment variable kullanımı bu dosyada yönetilebilir. Özellikle güvenli ortamda gizli bilgilerin doğrudan kod içine yazılmaması, environment variable üzerinden okunması güvenli yapılandırma prensibinin bir parçasıdır.

common/db.py dosyası SQLAlchemy veritabanı bağlantısını tanımlar. Flask uygulamaları bu ortak veritabanı nesnesini kullanarak veritabanı işlemlerini gerçekleştirir. Böylece veritabanı bağlantı yapısı tek bir noktadan yönetilir.

common/models.py dosyası SQLAlchemy model tanımlarını içerir. Kullanıcı ve item gibi veritabanı tabloları bu dosyada Python sınıfları olarak tanımlanır. Bu modeller hem güvenli hem de güvensiz API ortamlarında kullanılabilir. Böylece iki ortam aynı veri yapıları üzerinde çalışarak daha sağlıklı karşılaştırma yapılmasına imkân verir.

5.2.4. docker-compose.yml Dosyası

docker-compose.yml dosyası, projenin Docker ortamında çalıştırılmasını sağlayan temel yapılandırma dosyasıdır. Bu dosya sayesinde güvenli API, güvensiz API ve gerekli servisler tek komutla ayağa kaldırılabilir.

Docker Compose kullanımı sistemin taşınabilirliğini artırır. Kullanıcılar farklı bilgisayarlarda aynı ortamı tekrar kurabilir. Bu da eğitim amaçlı laboratuvarlarda büyük avantaj sağlar. Ayrıca güvenli ve güvensiz servislerin ayrı çalışma alanlarında yönetilmesi, sistemin daha kontrollü ve düzenli çalışmasına yardımcı olur.

5.2.5. requirements.txt Dosyası

requirements.txt dosyası, projenin ihtiyaç duyduğu Python kütüphanelerini içerir. Flask, SQLAlchemy, Flask-CORS, Flask-Limiter, bcrypt, PyJWT ve python-dotenv gibi bağımlılıklar bu dosya üzerinden kurulabilir.

Bu dosyanın bulunması, projenin kurulabilirliğini kolaylaştırır. Kullanıcılar gerekli bağımlılıkları tek komutla yükleyebilir:

```
pip install -r requirements.txt
```

5.2.6. Blueprint Mimarisi

Projede Flask Blueprint yapısı kullanılarak authentication işlemleri, kullanıcı yönetimi ve veri işleme senaryoları farklı route dosyaları içerisinde ayrıştırılmıştır. Bu mimari yaklaşım, backend kodunun tek bir dosyada karmaşık hale gelmesini engeller.

Blueprint yapısının projeye sağladığı avantajlar şunlardır:

- Kod okunabilirliğini artırır.
- Modüller arası sorumluluk ayrımı sağlar.
- Auth, users ve items işlemlerini birbirinden ayırır.
- Yeni endpoint eklemeyi kolaylaştırır.
- Güvenli ve güvensiz API ortamlarının karşılaştırılmasını kolaylaştırır.
- Test ve hata ayıklama süreçlerini daha yönetilebilir hale getirir.
- Yeni güvenlik senaryolarının sisteme eklenmesini kolaylaştırır.

Bu yapı sayesinde örneğin yalnızca authentication mekanizması üzerinde değişiklik yapılmak istendiğinde routes_auth.py dosyası düzenlenebilir. Kullanıcı yönetimiyle ilgili bir değişiklik yapılacağına routes_users.py, CRUD veya saldırı senaryolarıyla ilgili bir değişiklik yapılacağına ise routes_items.py üzerinde çalışılır. Bu da projeyi daha profesyonel ve sürdürülebilir hale getirir.

Sonuç olarak API Fortress'in dosya yapısı, saldırı ve savunma ortamlarını açık şekilde ayıran, ortak bileşenleri merkezi olarak yöneten ve Flask Blueprint mimarisiyle modülerliği destekleyen bir yapıda tasarlanmıştır. Bu organizasyon sayesinde sistemin bakım yapılabilirliği artmış, yeni güvenlik senaryolarının eklenmesi kolaylaşmış ve kullanıcıların güvenli-güvensiz API farklarını daha net biçimde analiz edebilmesi sağlanmıştır.

5.3. Veritabanı Tasarımı

Veritabanı katmanında SQLAlchemy ORM yapısı kullanılmıştır. ORM yaklaşımı sayesinde veritabanı işlemleri doğrudan SQL sorguları yerine Python nesneleri üzerinden gerçekleştirilebilmektedir. Bu yapı kod okunabilirliğini artırırken aynı zamanda veritabanı yönetimini daha güvenli hale getirmektedir.

Temel veri modeli olan User sınıfı aşağıdaki alanları içermektedir:

Alan	Açıklama
id	Birincil anahtar (otomatik artımlı tamsayı)
username	Kullanıcı adı (benzersiz, zorunlu)
email	E-posta adresi (benzersiz, zorunlu)
password	Bcrypt ile hashlenmiş parola
role	Kullanıcı rolü: 'user' veya 'admin'
created_at	Kayıt oluşturulma zaman damgası

5.4. REST API Endpoint Yapısı

Sistemdeki temel endpoint'ler ve bunların HTTP metodları aşağıdaki tabloda özetlenmektedir:

Endpoint	Metod	Açıklama
/api/auth/register	POST	Yeni kullanıcı kaydı
/api/auth/login	POST	Kullanıcı girişi ve token üretimi
/api/users	GET	Tüm kullanıcıları listeleme (admin)
/api/users/{id}	GET	Belirli kullanıcıyı görüntüleme
/api/users/{id}	PUT	Kullanıcı bilgilerini güncelleme
/api/users/{id}	DELETE	Kullanıcı silme (admin)
/api/items	GET/POST	Öğe listeleme ve ekleme
/api/items/{id}	PUT/DELETE	Öğe güncelleme ve silme

6. GÜVENSİZ API ORTAMI VE ZAFİYET SENARYOLARI

API Fortress sistemi içerisinde geliştirilen Vulnerable API ortamı, REST API güvenlik açıklarının uygulamalı olarak öğrenilmesini sağlamak amacıyla tasarlanmıştır. Bu ortamda çeşitli güvenlik açıkları bilinçli olarak sisteme entegre edilmiş; kullanıcıların saldırı senaryolarını kontrollü bir laboratuvar ortamında deneyimleyebilmesi hedeflenmiştir. Geliştirilen zafiyet senaryoları, OWASP API Security Top 10 kapsamında değerlendirilen modern API güvenlik problemlerini temel almaktadır.

Vulnerable API ortamının temel amacı, kullanıcıların yalnızca teorik güvenlik kavramlarını öğrenmesi değil; aynı zamanda bu güvenlik açıklarının gerçek sistemlerde nasıl ortaya çıktığını, saldırganların sistem davranışını nasıl manipüle ettiğini ve eksik güvenlik kontrollerinin ne tür sonuçlara yol açabileceğini uygulamalı olarak gözlemleyebilmesidir.

Bu ortamda bulunan endpoint'ler gerçek dünya REST API mimarisine benzer şekilde tasarlanmıştır. Kullanıcı kayıt işlemleri, kimlik doğrulama süreçleri, veri erişim mekanizmaları ve CRUD operasyonları normal bir backend sistemi mantığıyla çalışmaktadır. Ancak güvenlik kontrolleri bilinçli olarak eksik bırakılmış veya yanlış uygulanmıştır. Böylece kullanıcılar modern API sistemlerinde karşılaşılan zafiyetleri güvenli bir laboratuvar ortamında test edebilmektedir.

6.1. Mass Assignment

Mass Assignment zafiyeti, istemci tarafından gönderilen verilerin sunucu tarafında yeterli filtreleme yapılmadan doğrudan modele aktarılması sonucunda ortaya çıkan kritik bir güvenlik problemidir. Bu tür zafiyetlerde saldırgan, normalde erişememesi gereken model alanlarını HTTP isteği üzerinden manipüle ederek sistem davranışını değiştirebilir.

Modern REST API sistemlerinde kullanıcıdan gelen JSON verileri çoğu zaman doğrudan veri modellerine dönüştürülmektedir. Eğer geliştirici yalnızca izin verilmesi gereken alanları filtrelemezse, saldırgan ek alanlar göndererek sistem üzerinde yetki yükseltme veya kritik yapılandırma değişiklikleri gerçekleştirebilir.

Bu senaryoda kullanıcı kayıt endpoint'i güvenli olmayan şekilde tasarlanmıştır. Kullanıcıdan gelen JSON verileri herhangi bir filtreleme yapılmadan doğrudan User modeline aktarılmaktadır.

Güvensiz endpoint örneği aşağıdaki gibidir:

```
# Gvensiz kayt endpoint'i (routes_auth.py)

@auth_bp.route('/register', methods=['POST'])
def register():
    data = request.get_json()

    # Tehlike:
    # Gelen tm alanlar filtrelenmeden modele aktarılıyor

    new_user = User(**data)

    db.session.add(new_user)
    db.session.commit()

    return jsonify({'message': 'User created'}), 201
```

Bu yapı ilk bakışta işlevsel görünse de ciddi güvenlik riski oluşturmaktadır. Çünkü istemciden gelen tüm alanlar doğrudan modele aktarılmaktadır. Sistem yalnızca:

```
{
  "username": "enes",
  "password": "123456"
}
```

gibi alanlar beklerken saldırgan isteğe aşağıdaki alanı da ekleyebilir:

```
{
  "username": "attacker",
  "password": "123456",
  "role": "admin"
}
```

Bu durumda saldırgan kendi hesabını admin yetkisiyle oluşturabilir. Çünkü backend tarafında hangi alanların kabul edileceği sınırlandırılmamıştır.

6.1.1. Bu zafiyet özelliyle:

- kullanıcı rolleri,
- hesap durumu,
- erişim seviyeleri,
- ödeme bilgileri,
- sistem ayarları

gibi kritik alanların istemci tarafından değiştirilebilmesine neden olabilir.

Mass Assignment problemi modern API sistemlerinde oldukça yaygın görülen güvenlik açıklarından biridir. Özellikle ORM tabanlı backend sistemlerinde kullanıcıdan gelen verilerin doğrudan modele aktarılması ciddi risk oluşturmaktadır.

6.1.2. Bu senaryo sayesinde kullanıcılar:

- istemci verisinin neden filtrelenmesi gerektiğini,
- güvenli veri doğrulama mantığını,
- backend tarafında allowlist yaklaşımının önemini

uygulamalı olarak gözlemleyebilmektedir.

6.2. Broken Authentication

Broken Authentication zafiyeti, kullanıcı kimlik doğrulama süreçlerinde bulunan eksik veya hatalı güvenlik kontrolleri sonucunda ortaya çıkmaktadır. Kimlik doğrulama mekanizmaları modern backend sistemlerinin en kritik bileşenlerinden biridir. Bu mekanizmalardaki küçük bir hata bile saldırganların yetkisiz erişim elde etmesine neden olabilir.

API Fortress içerisindeki Broken Authentication senaryosunda iki farklı güvenlik problemi uygulanmıştır.

1. Parola doğrulama süreçlerinin eksik olması

Bazı durumlarda sistemin parola kontrolünü tam olarak gerçekleştirmemesi, saldırganların geçerli kullanıcı bilgileri olmadan sisteme erişebilmesine neden olabilir.

2. Gizli HTTP header ile authentication bypass

Bu senaryoda sisteme bilinçli olarak gizli bir arka kapı (backdoor) yerleştirilmiştir.

Güvensiz endpoint aşağıdaki gibidir:

```
# Gizli header ile authentication bypass

@auth_bp.route('/login', methods=['POST'])
def login():

    if request.headers.get('X-Admin-Override') == 'true':
        return jsonify({
            'token': generate_admin_token()
        }), 200

# Normal ak devam eder...
```

Bu senaryoda saldırgan normal kullanıcı bilgilerine ihtiyaç duymadan yalnızca aşağıdaki HTTP header'ını ekleyerek admin token'ı alabilmektedir:

```
X-Admin-Override: true
```

6.2.1. Bu yapı gerçek sistemlerde zaman zaman:

- debug mekanizmaları,
- test backdoor'ları,
- unutulmuş geliştirme kodları,
- yanlış yapılandırılmış admin bypass sistemleri

nedeniyle oluşabilmektedir.

6.2.2. Bu zafiyetin temel amacı:

- authentication mekanizmalarının neden kritik olduğunu,
- gizli bypass mekanizmalarının oluşturduğu riski,
- backend güvenlik kontrollerinin neden merkezi yönetilmesi gerektiğini

uygulamalı olarak göstermektir.

6.2.3. Broken Authentication problemleri özellikle:

- token manipülasyonu,
- session hijacking,
- brute force saldırıları,
- zayıf parola politikaları,
- gizli bypass mekanizmaları

gibi güvenlik problemleriyle birlikte modern API sistemlerinde en kritik tehditlerden biri olarak kabul edilmektedir.

6.3. IDOR / BOLA

IDOR (Insecure Direct Object Reference) ve BOLA (Broken Object Level Authorization), modern REST API sistemlerinde en yaygın ve en kritik güvenlik açıklarından biridir.

Bu zafiyetin temel nedeni, sistemin kullanıcıların erişmeye çalıştığı nesneler üzerinde yeterli authorization kontrolü yapmamasıdır.

API Fortress içerisindeki senaryoda kullanıcı detay endpoint'i güvenli olmayan şekilde tasarlanmıştır:

```
# Gvensiz kullanc detay endpoint'i

@users_bp.route('/users/<int:user_id>', methods=['GET'])
def get_user(user_id):

    # Tehlike:
    # Kimlik dorulama yok,
    # herhangi bir ID sorgulanabilir

    user = User.query.get(user_id)

    return jsonify(user.to_dict()), 200
```

6.3.1. Bu endpoint üzerinde:

- JWT doğrulaması bulunmamaktadır,
- kullanıcı kontrolü yapılmamaktadır,
- erişim yetkisi doğrulanmamaktadır.

Bu nedenle saldırgan URL içerisindeki kullanıcı ID'sini değiştirerek farklı kullanıcılara ait verilere erişebilir.

Örneğin:

```
GET /users/1
GET /users/2
GET /users/3
```

şeklinde farklı ID'ler denenerek sistemdeki diğer kullanıcı bilgileri görüntülenebilir.

6.3.2. Bu saldırı tipi özellikle:

- kullanıcı profilleri,

- sipariş sistemleri,
- banka işlemleri,
- müşteri kayıtları,
- sağlık sistemleri

gibi hassas veri içeren API sistemlerinde ciddi veri sızıntılarına neden olabilmektedir.

OWASP API Security Top 10 listesinde BOLA, en kritik API güvenlik problemlerinden biri olarak değerlendirilmektedir. Çünkü saldırgan çoğu zaman yalnızca URL parametresini değiştirerek yetkisiz veri erişimi sağlayabilmektedir.

6.3.3. Bu senaryo sayesinde kullanıcılar:

- object-level authorization mantığını, kullanıcı izolasyonunun önemini, erişim kontrolünün neden endpoint seviyesinde uygulanması gerektiğini uygulamalı olarak analiz edebilmektedir.

6.4. Broken Function Level Authorization

Broken Function Level Authorization zafiyeti, kullanıcıların erişmemesi gereken kritik sistem fonksiyonlarına ulaşabilmesi durumunda ortaya çıkmaktadır.

6.4.1. Bu problem genellikle:

- admin endpoint'lerinde,
- yönetim panellerinde,
- kritik CRUD işlemlerinde,
- rol tabanlı erişim kontrollerinde

yetersiz authorization uygulanması sonucunda oluşmaktadır.

API Fortress içerisindeki güvensiz kullanıcı silme endpoint'i aşağıdaki gibidir:

```
# Gvensiz kullanc silme endpoint'i

@users_bp.route('/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):

    # Tehlike:
    # Rol kontrol yok,
    # herkes herhangi kullancy silebilir

    user = User.query.get(user_id)

    db.session.delete(user)
    db.session.commit()

    return jsonify({
        'message': 'User deleted'
    }), 200
```

6.4.2. Bu endpoint üzerinde:

- kullanıcı rolü kontrol edilmemektedir,
- admin doğrulaması yapılmamaktadır,
- authorization middleware kullanılmamaktadır.

Bu nedenle normal bir kullanıcı aşağıdaki isteği göndererek başka kullanıcıları silebilir: Bu tür güvenlik açıkları gerçek sistemlerde:

- kullanıcı hesaplarının silinmesine,
- yönetici işlemlerinin ele geçirilmesine,
- veri kaybına,
- sistem bütünlüğünün bozulmasına

neden olabilir.

Broken Function Level Authorization zafiyetleri özellikle mikro servis mimarilerinde oldukça yaygın görülmektedir. Çünkü geliştiriciler çoğu zaman endpoint'in yalnızca frontend tarafından erişileceğini varsaymakta ve backend tarafında yeterli yetki kontrolü uygulamamaktadır.

6.4.3. Bu senaryo sayesinde kullanıcılar:

- rol tabanlı erişim kontrolünün önemini,
- admin endpoint'lerinin neden korunması gerektiğini,
- authorization süreçlerinin neden backend tarafında uygulanması gerektiğini

uygulamalı olarak öğrenebilmektedir.

Sonuç olarak Vulnerable API ortamı içerisinde geliştirilen Mass Assignment, Broken Authentication, IDOR/BOLA ve Broken Function Level Authorization senaryoları; modern REST API sistemlerinde yaygın olarak görülen kritik güvenlik problemlerini temsil etmektedir. Bu senaryolar sayesinde kullanıcılar yalnızca teorik güvenlik kavramlarını öğrenmekle kalmamakta; aynı zamanda gerçek dünya saldırı mantığını kontrollü laboratuvar ortamında uygulamalı olarak deneyimleyebilmektedir.

7. GÜVENLİ API ORTAMI VE SAVUNMA MEKANİZMALARI

API Fortress sistemi içerisinde geliştirilen Secure API ortamı, Vulnerable API ortamında bulunan güvenlik açıklarının modern savunma mekanizmaları kullanılarak engellenmesini amaçlamaktadır. Bu yapı sayesinde kullanıcıların yalnızca saldırı tekniklerini öğrenmesi değil, aynı zamanda güvenli backend sistemlerinin nasıl geliştirildiğini uygulamalı olarak inceleyebilmesi hedeflenmiştir.

Secure API ortamı, gerçek dünya REST API güvenlik prensiplerine uygun şekilde tasarlanmıştır. Sistem içerisinde kimlik doğrulama, yetkilendirme, saldırı filtreleme, rate limiting, parola güvenliği ve veri doğrulama mekanizmaları uygulanmıştır. Böylece kullanıcılar, güvenlik önlemlerinin backend mimarisi üzerindeki etkisini doğrudan analiz edebilmektedir.

Bu ortamın temel amacı yalnızca saldırıları engellemek değil; aynı zamanda modern güvenlik katmanlarının neden gerekli olduğunu ve eksik güvenlik kontrollerinin hangi risklere yol açtığını uygulamalı olarak göstermektir.

7.1. JWT Tabanlı Kimlik Doğrulama

Secure API ortamında kullanılan temel güvenlik mekanizmalarından biri JWT (JSON Web Token) tabanlı kimlik doğrulama sistemidir. JWT yapısı modern REST API mimarilerinde yaygın olarak kullanılan stateless authentication yaklaşımını temsil etmektedir.

Geleneksel oturum tabanlı sistemlerde kullanıcı oturum bilgileri sunucu belleğinde tutulurken, JWT yaklaşımında kullanıcı bilgileri imzalanmış bir token içerisinde taşınmaktadır. Bu yöntem sayesinde backend sistemi daha ölçeklenebilir hale gelmekte ve dağıtık sistem mimarilerine daha uygun bir yapı oluşmaktadır.

Kullanıcı sisteme başarılı şekilde giriş yaptığında backend tarafından imzalanmış bir JWT token oluşturulmaktadır. Bu token içerisinde genellikle:

- kullanıcı ID bilgisi,
- kullanıcı rolü,

- token oluşturma zamanı,
- token geçerlilik süresi

gibi bilgiler bulunmaktadır.

İstemci sonraki isteklerde bu token'ı HTTP Authorization header'ı içerisinde göndermektedir:

```
Authorization: Bearer <token>
```

Sunucu tarafında token doğrulama işlemleri aşağıdaki dekoratör yapısıyla gerçekleştirilmektedir. Bu yapı sayesinde sistem: token var mı kontrol eder, token imzasını doğrular, token süresinin dolup dolmadığını analiz eder, token içerisindeki kullanıcı bilgisini doğrular, geçersiz token'ları reddeder.

7.1.1. JWT doğrulama mekanizması sayesinde:

- yetkisiz erişimler engellenmekte,
- kullanıcı kimliği doğrulanmakta,
- güvenli endpoint erişimi sağlanmaktadır.

Ayrıca token süresinin kontrol edilmesi sayesinde eski veya çalınmış token'ların süresiz kullanılmasının önüne geçilmektedir.

7.1.2. Bu mekanizma özellikle:

- kullanıcı yönetim sistemleri,
- mobil uygulama backend'leri,
- mikro servis mimarileri,
- bulut tabanlı API sistemleri

gibi modern sistemlerde yaygın olarak kullanılmaktadır.

7.2. Rol Tabanlı Yetkilendirme (RBAC)

Secure API ortamında uygulanan bir diğer önemli savunma mekanizması RBAC (Role-Based Access Control) sistemidir.

Rol tabanlı yetkilendirme yapısının temel amacı, kullanıcıların yalnızca sahip oldukları yetkiler doğrultusunda işlem yapabilmesini sağlamaktır. Böylece kritik işlemlerin yalnızca yetkili kullanıcılar tarafından gerçekleştirilmesi mümkün olmaktadır.

Sistem içerisinde temel olarak iki farklı kullanıcı rolü bulunmaktadır:

- user
- admin

Her rol farklı erişim seviyelerine sahiptir.

7.2.1. Örneğin:

- normal kullanıcılar yalnızca kendi işlemlerini gerçekleştirebilir,
- admin kullanıcılar kullanıcı yönetimi gibi kritik işlemlere erişebilir.

7.2.2. Bu yapı özelliklerle:

- Broken Access Control,
- Broken Function Level Authorization,
- BOLA/IDOR

gibi modern API güvenlik problemlerini önlemek açısından kritik öneme sahiptir.

Mass Assignment zafiyetini engellemek amacıyla istemci tarafından gönderilen kritik alanlar filtrelenmiştir.

7.2.3. Bu yapıda:

- istemciden gelen role alanı tamamen yok sayılmaktadır,
- kullanıcı rolü backend tarafından sabit olarak belirlenmektedir,
- kritik güvenlik alanlarının istemci tarafından değiştirilmesi engellenmektedir.

Böylece saldırgan aşağıdaki isteği gönderse bile:

```
{  "username": "attacker",  "password": "123456",  "role": "admin"}
```

sistem kullanıcıyı yalnızca normal user rolüyle oluşturacaktır.

Bu yaklaşım modern güvenli backend sistemlerinde kullanılan:

- allowlist validation,
- server-side authorization,
- secure object creation

prensiplerini temsil etmektedir.

7.3. Rate Limiting

API Fortress sistemi içerisinde brute force ve servis yoğunlaştırma saldırılarına karşı Flask-Limiter kullanılarak rate limiting mekanizması uygulanmıştır.

7.3.1. Rate limiting mekanizmasının temel amacı:

- istemcilerin aşırı istek göndermesini engellemek,
- brute force saldırılarını zorlaştırmak,
- sistem kaynaklarını korumak,
- spam istekleri azaltmaktır.

Sistem genelinde varsayılan rate limit kuralları uygulanmıştır:

```
from flask_limiter import Limiterfrom flask_limiter.util import get_remote_add
```

Bu yapı sayesinde istemcilerin: günlük, saatlik, endpoint bazlı istek sayıları kontrol edilmektedir.

Özellikle login endpoint'inde daha sıkı limit uygulanmıştır:

```
@auth_bp.route('/login', methods=['POST'])@limiter.limit('5 per minute')def lo
```

Bu yapı sayesinde kullanıcı dakikada yalnızca 5 giriş denemesi yapabilmektedir.

Bu mekanizma: brute force parola saldırılarını, otomatik bot aktivitelerini, credential stuffing saldırılarını önemli ölçüde zorlaştırmaktadır.

7.3.2. Gerçek dünya sistemlerinde rate limiting:

- API gateway'lerde,
- CDN katmanlarında,
- load balancer sistemlerinde,
- authentication servislerinde

kritik savunma mekanizmalarından biri olarak kullanılmaktadır.

7.4. Web Application Firewall (WAF)

API Fortress sisteminde zararlı isteklerin filtrelenebilmesi amacıyla regex tabanlı Web Application Firewall (WAF) mekanizması geliştirilmiştir.

7.4.1. WAF sisteminin temel amacı:

- zararlı payload'ları analiz etmek,
- şüpheli istekleri filtrelemek,

- injection saldırılarını engellemek,
- istemci verilerini kontrol etmektir.

Sistem içerisinde belirli saldırı pattern'leri tanımlanmıştır:

```
WAF_PATTERNS = [ r'(\textbackslash{\}bor\textbackslash{\}s+1=1\textbackslash{\}
```

7.4.2. Bu pattern'ler:

- SQL Injection,
- XSS,
- zararlı script içerikleri,
- veri manipülasyon girişimleri

gibi saldırıları tespit etmeye çalışmaktadır.

İstek kontrol mekanizması şu şekilde çalışmaktadır:

```
def waf_check(data: str) -> bool:     for pattern in WAF_PATTERNS:         if re
```

Sistem:

1. gelen isteği analiz eder,
2. pattern listesiyle karşılaştırır,
3. şüpheli içerik tespit edilirse isteği reddeder.

Örneğin aşağıdaki payload'lar engellenebilir:

```
OR 1=1UNION SELECT<script>alert(1)</script>DROP TABLE usersjavascript:
```

Bu yapı gerçek kurumsal WAF sistemlerinin basitleştirilmiş laboratuvar simülasyonudur.

7.4.3. Amaç:

- input filtering mantığını göstermek,
- saldırı tespit süreçlerini öğretmek,
- backend güvenlik katmanlarının çalışma prensibini açıklamaktır.

7.5. Güvenli Parola Hashleme ve Input Validation

Secure API ortamında kullanıcı parolalarının düz metin olarak saklanması tamamen engellenmiştir. Bunun yerine bcrypt tabanlı parola hashleme mekanizması uygulanmıştır.

7.5.1. Bcrypt algoritmasının kullanılmasının temel amacı:

- parola sızıntılarının etkisini azaltmak,
- brute force saldırı maliyetini artırmak,
- rainbow table saldırılarını zorlaştırmaktır.

Kullanıcı kaydı sırasında parola doğrudan veritabanına yazılmaz:

```
bcrypt.generate_password_hash( data.get('password') )
```

7.5.2. Bu işlem sonucunda:

- geri döndürülemez hash değeri oluşturulur,
- her kullanıcı için farklı salt üretilir,
- parola güvenliği artırılır.

Böylece veritabanı sızdırılsa bile kullanıcı şifreleri doğrudan ele geçirilemez.

Ayrıca sistem içerisinde input validation mekanizmaları uygulanmıştır.

7.5.3. Bu yapı sayesinde:

- eksik alanlar,
- geçersiz veri tipleri,
- kısa parolalar,
- hatalı email formatları,
- beklenmeyen veri girişleri

kontrol edilmektedir.

7.5.4. Örneğin:

- boş kullanıcı adı,
- geçersiz email,
- 3 karakterlik parola

gibi durumlar sistem tarafından reddedilmektedir.

7.5.5. Validation mekanizmasının temel amacı:

- sistem kararlılığını korumak,
- zararlı veri girişlerini azaltmak,
- güvenli veri işleme sağlamak,
- istemci hatalarını yönetmektir.

Ayrıca kullanıcıya anlamlı hata mesajları döndürülmektedir:

```
{  "error": "Password must be at least 8 characters"}
```

7.5.6. Bu yaklaşım:

- kullanıcı deneyimini artırmakta,
- backend hata yönetimini güçlendirmekte,
- veri bütünlüğünü korumaktadır.

Sonuç olarak Secure API ortamı içerisinde uygulanan JWT tabanlı authentication, RBAC yetkilendirme sistemi, rate limiting mekanizması, regex tabanlı WAF yapısı, bcrypt parola hashleme sistemi ve input validation kontrolleri; modern REST API güvenlik prensiplerini uygulamalı biçimde göstermektedir. Bu yapı sayesinde kullanıcılar yalnızca saldırı tekniklerini değil; aynı zamanda güvenli backend mimarisinin nasıl tasarlanması gerektiğini de uygulamalı olarak inceleyebilmektedir.

8. UYGULAMA GELİŞTİRME SÜRECİ

8.1. Geliştirme Ortamı

Proje geliştirme sürecinde kullanılan teknoloji yığını aşağıdaki tabloda özetlenmektedir:

Teknoloji	Detay
Programlama Dili	Python 3.11
Web Framework	Flask 3.x
ORM	SQLAlchemy 2.x
Veritabanı	SQLite (geliştirme) / PostgreSQL (üretim)
Kimlik Doğrulama	PyJWT
Parola Hashleme	Flask-Bcrypt
Rate Limiting	Flask-Limiter
Ortam Yönetimi	python-dotenv
Konteyner	Docker & Docker Compose
Test Araçları	Postman, curl, Python requests
Versiyon Kontrolü	Git

8.2. Docker Konteyner Yapısı

API Fortress sisteminin farklı platformlarda kolayca çalıştırılabilmesi amacıyla Docker tabanlı konteyner mimarisi tercih edilmiştir. Bu yapı sayesinde sistem bileşenleri izole şekilde çalıştırılmakta ve laboratuvar ortamının taşınabilirliği artırılmaktadır. Böylece geliştirme, test ve dağıtım süreçleri daha yönetilebilir hale getirilmektedir.

```
# docker-compose.yml
version: '3.8'
services:
  insecure_api:
    build: ./insecure_api
    ports:
```

```
- '5000:5000'
environment:
  - FLASK_ENV=development

secure_api:
  build: ./secure_api
  ports:
    - '5001:5001'
  environment:
    - SECRET_KEY=${SECRET_KEY}
    - DATABASE_URL=${DATABASE_URL}
```

Güvenli ortamda SECRET_KEY ve DATABASE_URL gibi hassas bilgiler .env dosyası üzerinden environment variable olarak yönetilmektedir. Bu yaklaşım, gizli bilgilerin kaynak kod içine gömülmesini engelleyen kritik bir güvenlik önlemidir.

8.3. Kod Organizasyonu

API Fortress projesinde kod organizasyonu, sistemin okunabilirliğini, sürdürülebilirliğini ve genişletilebilirliğini artırmak amacıyla modüler bir yapıda tasarlanmıştır. Kod tabanı; kimlik doğrulama, kullanıcı yönetimi ve öge yönetimi gibi özellik alanlarına göre ayrıştırılmıştır. Bu sayede her modül kendi sorumluluk alanına sahip olmuş, tek bir dosya içerisinde karmaşık ve yönetilmesi zor bir backend yapısı oluşması engellenmiştir.

Projede temel olarak şu modüller ayrım uygulanmıştır:

routes_auth.py	→ Kayıt, giriş ve token işlemleri
routes_users.py	→ Kullanıcı listeleme, detay, silme ve yetki işlemleri
routes_items.py	→ CRUD işlemleri ve veri yönetimi
models.py	→ Veritabanı model tanımları
db.py	→ SQLAlchemy veritabanı bağlantısı
config.py	→ Ortam değişkenleri ve yapılandırma ayarları

Bu yapı sayesinde authentication işlemleri kullanıcı yönetiminden, kullanıcı yönetimi de öge yönetiminden ayrılmıştır. Örneğin giriş ve kayıt işlemlerinde bir değişiklik yapılmak istendiğinde yalnızca routes_auth.py dosyası üzerinde çalışmak yeterli olur.

Kullanıcı listeleme veya silme gibi işlemler `routes_users.py` içerisinde, CRUD işlemleri ise `routes_items.py` içerisinde yönetilir. Bu ayırım, kodun bakımını kolaylaştırır ve hata ayıklama sürecini daha kontrollü hale getirir.

Kod tabanı yalnızca dosyalara ayrılmakla kalmamış; aynı zamanda denetleyici, servis ve veri erişim mantığı açısından da düzenli bir yapıda ele alınmıştır. Denetleyici katmanı yani route dosyaları, istemciden gelen HTTP isteklerini karşılayan bölümdür. Bu katman gelen isteği alır, gerekli güvenlik kontrollerini çağırır, veritabanı işlemlerini başlatır ve sonucu JSON formatında istemciye döndürür.

Servis mantığı ise uygulamanın iş kurallarını temsil eder. Örneğin kullanıcı oluşturulurken rol bilgisinin backend tarafından atanması, parola hashleme işleminin yapılması, token üretimi veya yetki kontrolü gibi işlemler bu iş mantığının parçasıdır. Veri erişim katmanı ise SQLAlchemy modelleri ve veritabanı bağlantısı üzerinden yürütülür. Böylece sistemde HTTP isteği alma, iş kuralı uygulama ve veritabanı işlemi yapma görevleri birbirinden ayrılmıştır.

Her iki API ortamı da, yani güvenli ve güvensiz backend yapıları, benzer klasör ve dosya organizasyonunu paylaşmaktadır. Bu tercih bilinçli olarak yapılmıştır. Çünkü projenin temel amacı, aynı REST API işlevlerinin güvenli ve güvensiz uygulamalarını karşılaştırmalı biçimde gösterebilmektir. Kullanıcılar `insecure_api` klasöründeki bir endpoint ile `secure_api` klasöründeki karşılığını inceleyerek güvenlik mekanizmalarının kod seviyesinde nasıl fark oluşturduğunu görebilmektedir.

Örneğin güvensiz ortamda kayıt işlemi gelen tüm JSON verilerini doğrudan modele aktarırken, güvenli ortamda yalnızca izin verilen alanlar alınır ve rol bilgisi sunucu tarafından sabitlenir. Benzer şekilde güvensiz kullanıcı silme endpoint'inde rol kontrolü bulunmazken, güvenli ortamda bu işlem yalnızca admin yetkisine sahip kullanıcılar tarafından gerçekleştirilebilir. Bu paralel dosya yapısı, saldırı ve savunma farkının doğrudan kod üzerinden anlaşılmasını kolaylaştırır.

Projede Flask Blueprint mimarisi kullanılmıştır. Blueprint yapısı, Flask uygulamalarında route'ların modüler şekilde organize edilmesini sağlar. Bu sayede tüm endpoint'lerin tek bir `app.py` dosyasında toplanması yerine, işlevlerine göre ayrı dosyalara bölünmesi mümkün hale gelmiştir. `app.py` dosyası uygulamanın başlangıç noktası olarak görev yapar ve ilgili Blueprint modüllerini uygulamaya kaydeder. Böylece ana uygulama dosyası sade kalır, endpoint yönetimi ise ilgili modüllerde yapılır.

Bu yaklaşımın projeye sağladığı başlıca avantajlar şunlardır:

- Kod okunabilirliği artar.
- Hata ayıklama daha kolay hale gelir.
- Yeni endpoint eklemek kolaylaşır.
- Güvenli ve güvensiz ortamlar karşılaştırmalı incelenebilir.
- Her dosya yalnızca kendi sorumluluğuna odaklanır.
- Güvenlik senaryoları sisteme daha kolay eklenir.
- Proje büyüdüğünde kod karmaşası azalır.

Kod organizasyonunun bir diğer önemli parçası common klasörüdür. Bu klasör, her iki API ortamı tarafından ortak kullanılan bileşenleri içerir. Veritabanı bağlantısı, model tanımları ve yapılandırma ayarları ortak klasör altında tutulduğu için kod tekrarının önüne geçilmiştir. Böylece hem güvenli hem de güvensiz API ortamları aynı veri modelleri üzerinden çalışabilmekte ve karşılaştırmalı analiz daha tutarlı hale gelmektedir.

CORS yapılandırması da kod organizasyonunda önemli bir yer tutmaktadır. CORS (Cross-Origin Resource Sharing), farklı kaynaklardan gelen istemci isteklerinin API'ye erişip erişemeyeceğini belirleyen bir tarayıcı güvenlik mekanizmasıdır. Projede Flask-CORS kütüphanesi kullanılarak CORS ayarları yapılandırılmıştır. Bu sayede farklı portta veya farklı domain üzerinde çalışan frontend uygulamalarının backend API'ye kontrollü şekilde erişebilmesi sağlanmıştır.

Örneğin frontend uygulaması localhost:3000 üzerinde, Flask backend ise localhost:5000 üzerinde çalışıyorsa tarayıcı normalde bu isteği cross-origin olarak değerlendirir. Flask-CORS yapılandırması sayesinde bu iletişim izinli hale getirilir. Böylece frontend ile backend arasındaki REST API haberleşmesi sorunsuz biçimde gerçekleştirilebilir.

Güvensiz API ortamında CORS ayarları daha geniş tutulabilir. Bu durum eğitim amacıyla yanlış yapılandırılmış CORS politikalarının potansiyel risklerini göstermek için kullanılabilir. Güvenli API ortamında ise CORS izin verilen kaynak listesi sınırlandırılmıştır. Böylece yalnızca güvenilir frontend adreslerinin API'ye erişmesine izin verilir. Bu yaklaşım, üretim ortamlarında uygulanması gereken daha güvenli CORS politikasını temsil etmektedir.

8.3.1. Güvenli CORS yapılandırmasının amacı şudur:

- Her kaynaktan gelen isteği kabul etmemek,
- API erişimini güvenilir domainlerle sınırlandırmak,
- Yetkisiz frontend kaynaklarından gelen istekleri azaltmak,
- Tarayıcı tabanlı güvenlik risklerini düşürmek.

Sonuç olarak API Fortress projesinde kod organizasyonu; modülerlik, karşılaştırılabilirlik ve güvenli geliştirme prensipleri dikkate alınarak tasarlanmıştır. Flask Blueprint yapısı sayesinde route dosyaları işlevlerine göre ayrılmış, ortak bileşenler common klasöründe toplanmış, güvenli ve güvensiz API ortamları paralel dosya yapısıyla düzenlenmiş ve CORS yapılandırması güvenli ortamda sınırlandırılmıştır. Bu yapı, projenin hem eğitim amaçlı incelenmesini kolaylaştırmakta hem de modern backend geliştirme standartlarına uygun bir mimari sunmaktadır.

9. TEST SÜRECİ VE CTF SENARYOLARI

9.1. Test Stratejisi

API Fortress sisteminin geliştirme sürecinin ardından hem Vulnerable API hem de Secure API ortamı üzerinde çeşitli güvenlik ve işlevsellik testleri gerçekleştirilmiştir. Testlerin temel amacı, güvensiz ortamda oluşturulan zafiyetlerin kontrollü şekilde sömürülebilir olduğunu doğrulamak ve güvenli ortamda geliştirilen savunma mekanizmalarının doğru çalıştığını göstermektir.

9.1.1. Test sürecinde üç temel yöntem kullanılmıştır:

- Postman koleksiyonları
- curl komutları
- Python tabanlı otomatik test betikleri

Postman kullanılarak kullanıcı kayıt işlemleri, giriş süreçleri, JWT token doğrulamaları, CRUD işlemleri ve saldırı senaryoları manuel olarak test edilmiştir. SQL Injection, XSS, IDOR/BOLA ve Mass Assignment gibi saldırılar için özel request koleksiyonları hazırlanmıştır. Böylece aynı test senaryoları tekrar çalıştırılarak güvenli ve güvensiz ortamlar karşılaştırılmıştır.

curl komutları özellikle terminal üzerinden hızlı endpoint testleri gerçekleştirmek amacıyla kullanılmıştır. Authorization header kontrolleri, token doğrulama süreçleri ve rate limiting davranışları curl üzerinden analiz edilmiştir. Özellikle brute force testlerinde kısa süre içerisinde çok sayıda login isteği gönderilerek sistemin rate limiting mekanizması doğrulanmıştır.

Python test betikleri ise daha otomatik saldırı senaryoları için geliştirilmiştir. Bu betikler sayesinde ardışık kullanıcı ID'leri üzerinde IDOR testleri, otomatik payload denemeleri ve endpoint taramaları yapılmıştır. Ayrıca WAF mekanizmasının zararlı girdilere verdiği tepkiler analiz edilmiştir.

9.1.2. Vulnerable API ortamında gerçekleştirilen testlerde:

- Mass Assignment ile admin yetkisi kazanılabildiği,
- Authentication bypass yapılabildiği,
- Yetkisiz veri erişiminin mümkün olduğu,
- Rol kontrolü eksikliklerinin bulunduğu

doğrulanmıştır.

9.1.3. Secure API ortamında yapılan testlerde ise:

- JWT doğrulama sisteminin çalıştığı,
- Yetkisiz erişimlerin engellendiği,
- Rate limiting mekanizmasının brute force saldırılarını sınırlandığı,
- WAF sisteminin SQL Injection ve XSS payload'larını filtrelediği,
- RBAC yapısının admin işlemlerini koruduğu

gözlemlenmiştir.

Gerçekleştirilen testler sonucunda API Fortress sisteminin hem saldırı senaryolarını uygulamalı olarak gösterebildiği hem de modern savunma mekanizmalarının etkinliğini başarılı şekilde sergileyebildiği doğrulanmıştır.

9.2. Zafiyet Testleri

Her zafiyet senaryosu için iki aşamalı test yapılmıştır: (1) Güvensiz ortamda saldırının başarılı olduğunun doğrulanması, (2) Güvenli ortamda aynı saldırının engellendiğinin doğrulanması.

Senaryo	Saldırı Tekniği	Güvensiz Sonuç	Güvenli Sonuç
Mass Assignment	role: admin ile kayıt	Admin yetkisi kazanıldı (evet)	HTTP 400 – engellendi (evet)
Broken Auth	X-Admin-Override header	Token olmadan giriş (evet)	HTTP 401 – engellendi (evet)
IDOR / BOLA	Farklı user_id sorgusu	Başka kullanıcı verisi (evet)	HTTP 403 – engellendi (evet)
BFLA	Normal kullanıcı ile DELETE	Kullanıcı silindi (evet)	HTTP 403 – engellendi (evet)
Brute Force	100 istek / dakika	Sınırsız deneme (evet)	HTTP 429 – engellendi (evet)
SQL Injection	OR 1=1 payload	Veri sızdırıldı (evet)	WAF engelledi (evet)

9.3. Test Sonuçları

Gerçekleştirilen testler sonucunda Vulnerable API ortamında oluşturulan tüm temel güvenlik açıklarının başarıyla sömürülebildiği doğrulanmıştır. Mass Assignment, Broken Authentication, IDOR/BOLA ve Broken Function Level Authorization senaryolarında saldırganın yetkisiz erişim elde edebildiği, kullanıcı rollerini manipüle edebildiği ve farklı kullanıcılara ait verilere erişebildiği gözlemlenmiştir. Bu sonuçlar, güvenlik kontrollerinin eksik bırakıldığı REST API sistemlerinde oluşabilecek riskleri uygulamalı olarak göstermiştir.

Secure API ortamında ise geliştirilen savunma mekanizmalarının saldırıları büyük ölçüde engellediği görülmüştür. JWT tabanlı kimlik doğrulama sistemi sayesinde yetkisiz kullanıcı erişimleri engellenmiş, RBAC mekanizması ile kritik işlemler yalnızca yetkili kullanıcılarla sınırlandırılmıştır. Ayrıca rate limiting sistemi brute force giriş denemelerini sınırlandırırken, WAF filtreleme mekanizması standart SQL Injection ve XSS payload'larını başarılı şekilde tespit ederek zararlı istekleri engellemiştir. Özellikle authorization kontrolleri, request filtering yapıları ve rate limiting mekanizmalarının sistem güvenliğini önemli ölçüde artırdığı gözlemlenmiştir.

Test sürecinde tespit edilen önemli bir kısmi yetersizlik ise regex tabanlı WAF kurallarının yalnızca standart saldırı payload'larını hedef almasıdır. URL-encoded, Base64

encoded veya farklı obfuscation teknikleri kullanılarak gizlenen bazı payload'ların mevcut filtreleme mekanizmasını aşabileceği görülmüştür. Bu durum, regex tabanlı temel WAF sistemlerinin gelişmiş saldırılara karşı tek başına yeterli olmadığını göstermektedir. Bu nedenle gelecekteki sürümlerde çok katmanlı filtreleme mekanizmaları, gelişmiş payload normalizasyonu ve davranış tabanlı saldırı analizi gibi ek güvenlik önlemlerinin sisteme entegre edilmesi planlanmaktadır.

10. SONUÇLAR VE ÖNERİLER

Bu proje kapsamında geliştirilen API Fortress sistemi, REST API güvenliği alanında uygulamalı eğitim sağlayan saldırı ve savunma simülasyon laboratuvarı olarak başarıyla tamamlanmıştır. Sistem içerisinde hem güvenlik açıkları içeren Vulnerable API ortamı hem de modern savunma mekanizmalarıyla güçlendirilmiş Secure API ortamı birlikte geliştirilmiştir. Böylece kullanıcıların yalnızca teorik güvenlik bilgisi edinmesi değil; aynı zamanda gerçek saldırı senaryolarını uygulamalı olarak deneyimleyebilmesi ve güvenli backend mimarisinin nasıl çalıştığını doğrudan inceleyebilmesi sağlanmıştır.

Proje kapsamında OWASP API Security Top 10 içerisinde yer alan modern API güvenlik problemleri temel alınmıştır. Özellikle Mass Assignment, Broken Authentication, IDOR/BOLA ve Broken Function Level Authorization gibi kritik güvenlik açıkları laboratuvar ortamına başarılı şekilde entegre edilmiştir. Bu sayede kullanıcılar yetkisiz erişim, role escalation, authentication bypass ve object-level authorization problemleri gibi gerçek dünya saldırı senaryolarını kontrollü bir ortamda analiz edebilmiştir.

Güvenli API ortamında ise modern REST API güvenlik prensiplerini temsil eden çeşitli savunma mekanizmaları geliştirilmiştir. JWT tabanlı kimlik doğrulama sistemi ile kullanıcı doğrulama süreçleri güvenli hale getirilmiş, rol tabanlı yetkilendirme (RBAC) sistemi ile kullanıcı ve yönetici işlemleri birbirinden ayrılmıştır. Ayrıca Flask-Limiter kullanılarak brute force saldırılarına karşı rate limiting mekanizması uygulanmış, regex tabanlı WAF sistemi ile SQL Injection ve XSS payload'larının filtrelenmesi sağlanmıştır. Kullanıcı parolalarının bcrypt algoritmasıyla hashlenmesi ve input validation süreçlerinin uygulanması sayesinde veri güvenliği artırılmıştır.

Gerçekleştirilen testler sonucunda Vulnerable API ortamında oluşturulan zafiyetlerin başarıyla sömürülebildiği doğrulanmıştır. Secure API ortamında ise geliştirilen savunma mekanizmalarının büyük ölçüde etkili olduğu gözlemlenmiştir. Özellikle JWT doğrulama sistemi, authorization kontrolleri, request filtering yapıları ve rate limiting mekanizmalarının sistem güvenliğini önemli ölçüde artırdığı görülmüştür. Böylece proje, saldırı ve savunma mekanizmalarının aynı sistem üzerinde karşılaştırmalı biçimde incelenebildiği uygulamalı bir laboratuvar ortamı sunmuştur.

Bu çalışma ile birlikte elde edilen en önemli katkılardan biri, REST API güvenliği üzerine doğrudan odaklanan özgün bir eğitim platformunun geliştirilmiş olmasıdır. Mevcut birçok güvenlik laboratuvarı genel web güvenliğine yoğunlaşırken API Fortress özellikle REST API

zafiyetlerini hedef almış ve bu açıkları saldırı ve savunma boyutlarıyla birlikte sunmuştur. Böylece kullanıcılar modern backend sistemlerinde karşılaşılan güvenlik problemlerini daha gerçekçi biçimde analiz edebilmiştir.

Projenin önemli katkılarından biri de Docker tabanlı taşınabilir laboratuvar mimarisidir. Konteyner tabanlı yapı sayesinde sistem farklı işletim sistemlerinde ve geliştirme ortamlarında kolaylıkla çalıştırılabilmektedir. Bu yaklaşım hem laboratuvar ortamının tekrar üretilebilirliğini artırmış hem de eğitim erişilebilirliğini kolaylaştırmıştır. Kullanıcılar karmaşık kurulum süreçleriyle uğraşmadan sistemi çalıştırarak güvenlik testlerini gerçekleştirebilmektedir.

CTF (Capture The Flag) benzeri saldırı senaryolarının sisteme entegre edilmesi de aktif öğrenme deneyimini destekleyen önemli bir unsur olmuştur. Kullanıcılar yalnızca teorik bilgi okumak yerine gerçek güvenlik açıklarını çözmeye çalışarak sistem davranışını analiz edebilmiştir. Bu yaklaşım, güvenlik farkındalığının artırılması ve öğrenme sürecinin daha kalıcı hale gelmesi açısından önemli avantaj sağlamıştır.

Ayrıca güvenli ve güvensiz API ortamlarının aynı sistem içerisinde birlikte sunulması sayesinde karşılaştırmalı analiz imkânı oluşturulmuştur. Kullanıcılar aynı endpoint'in güvensiz sürümünde saldırının neden başarılı olduğunu, güvenli sürümünde ise hangi savunma mekanizmaları nedeniyle engellendiğini doğrudan inceleyebilmiştir. Bu yaklaşım modern güvenli yazılım geliştirme prensiplerinin daha anlaşılır hale gelmesini sağlamıştır.

Proje sürecinde bazı geliştirme alanları da tespit edilmiştir. Özellikle regex tabanlı WAF yapısının yalnızca standart payload'ları filtreleyebilmesi, gelişmiş encoded veya obfuscated saldırılar karşısında ek güvenlik katmanlarına ihtiyaç duyulduğunu göstermiştir. Ayrıca sistem şu anda temel REST API güvenlik senaryolarına odaklanmaktadır. Gelecek çalışmalarda:

- gelişmiş WAF mekanizmaları,
- davranış tabanlı saldırı analizi,
- yapay zekâ destekli anomali tespiti,
- API Gateway entegrasyonu,
- merkezi log yönetimi,
- SIEM entegrasyonu,

- gerçek zamanlı saldırı izleme sistemleri

gibi daha ileri güvenlik bileşenlerinin sisteme eklenmesi planlanmaktadır.

Sonuç olarak API Fortress projesi, REST API güvenliğini uygulamalı olarak öğretmeyi amaçlayan kapsamlı bir saldırı ve savunma laboratuvarı olarak başarılı şekilde geliştirilmiştir. Proje hem modern REST API mimarisini hem de güncel güvenlik mekanizmalarını uygulamalı biçimde göstermekte; güvenli yazılım geliştirme farkındalığının artırılmasına katkı sağlamaktadır. Bu yönüyle sistem, hem eğitimsel hem de teknik açıdan REST API güvenliği alanında kullanılabilecek taşınabilir ve genişletilebilir bir laboratuvar platformu sunmaktadır.

10.1. Gelecek Çalışmalar

Sistemin genişletilebilirliği gözetilerek aşağıdaki iyileştirmeler planlanmaktadır:

- Yeni OWASP API Security senaryolarının (API6-API10) laboratuvara eklenmesi.
- Gerçek zamanlı saldırı analizi ve görselleştirme modülü geliştirilmesi.
- Farklı zorluk seviyelerine sahip CTF senaryolarının oluşturulması ve otomatik puanlama sistemi.
- Kullanıcı ilerleme takip sistemi ve öğrenme yolu yönetimi.
- Makine öğrenmesi tabanlı anomali tespiti ve davranışsal WAF modülü.
- Frontend kullanıcı arayüzünün geliştirilmesi ve görsel dashboard entegrasyonu.

KAYNAKLAR

- [1] OWASP Foundation. (2023). OWASP API Security Top 10 2023. OWASP API Security Project.
- [2] Roy Thomas Fielding (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.

- [3] Michael Jones, John Bradley, & Nat Sakimura (2015). RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force (IETF).
- [4] Flask Documentation. (2024). Flask Web Framework.
- [5] SQLAlchemy Documentation. (2024). SQLAlchemy ORM and Database Toolkit.
- [6] Docker. (2024). Docker Documentation.
- [7] M. Jensen, Nils Gruschka, & Ralf Herkenhöner (2019). A Survey of Attacks on Web Services and Countermeasures. *International Journal of Web Information Systems*, 15(3), 224–246.
- [8] PortSwigger. (2024). Web Security Academy.
- [9] OWASP Foundation. (2023). OWASP Juice Shop Project.
- [10] Carlos Vargas, Mohamed Emam, & Nils Ruehlen (2020). REST API Security Vulnerabilities in Practice: An Empirical Study. In *Proceedings of the IEEE International Conference on Software Quality, Reliability and Security*, 87–97.
- [11] OWASP Foundation. (2024). OWASP API Security Project Documentation.
- [12] JSON Web Token Introduction. (2024). JWT.io Documentation.
- [13] Auth0. (2024). JSON Web Tokens Documentation.
- [14] PortSwigger. (2024). API Testing and OWASP Top 10 Vulnerabilities.
- [15] OWASP Foundation. (2023). OWASP API Security Top 10 – 2023 Edition.

ÖZGEÇMİŞ

Mehmet Enes DENİZ

Adı Soyadı	Mehmet Enes DENİZ
Uyruğu	T.C.
Doğum Yeri ve Tarihi	...
E-posta	220205039@ostimteknik.edu.tr

Eğitim Derecesi	Kurum / Yıl
Lise	...
Lisans	Yazılım Mühendisliği – OSTİM Teknik Üniversitesi (Devam ediyor)

Uzmanlık Alanı 1	REST API Geliştirme ve Güvenliği
Uzmanlık Alanı 2	Python / Flask Backend Geliştirme
Yabancı Dil	İngilizce – B2

Ege KILINÇ

Adı Soyadı	Ege KILINÇ
Uyruğu	T.C.
Doğum Yeri ve Tarihi	...
E-posta	220205040@ostimteknik.edu.tr

Eğitim Derecesi	Kurum / Yıl
Lise	...
Lisans	Yazılım Mühendisliği – OSTİM Teknik Üniversitesi (Devam ediyor)

Uzmanlık Alanı 1	Siber Güvenlik ve Etik Hacking
Uzmanlık Alanı 2	Docker ve Konteyner Mimarileri
Yabancı Dil	İngilizce – B1

11. Ek-1: Kod Gösterimi

11.1. app.py – Güvenli API Başlangıç Noktası

```
from flask import Flask
from flask_cors import CORS
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
from dotenv import load_dotenv
from common.db import db
from routes_auth import auth_bp
from routes_users import users_bp
from routes_items import items_bp
import os

load_dotenv()
app = Flask(__name__)
app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY')
app.config['SQLALCHEMY_DATABASE_URI'] = os.environ.get('DATABASE_URL')

CORS(app, origins=['http://localhost:3000'])
limiter = Limiter(key_func=get_remote_address, app=app,
                  default_limits=['200 per day', '50 per hour'])

db.init_app(app)
app.register_blueprint(auth_bp, url_prefix='/api/auth')
app.register_blueprint(users_bp, url_prefix='/api/users')
app.register_blueprint(items_bp, url_prefix='/api/items')

if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(host='0.0.0.0', port=5001, debug=False)
```

11.2. models.py – Kullanıcı Modeli

```
from common.db import db
from datetime import datetime

class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(255), nullable=False)
    role = db.Column(db.String(20), default='user')
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    def to_dict(self):
        return {
            'id': self.id,
            'username': self.username,
            'email': self.email,
            'role': self.role,
            'created_at': self.created_at.isoformat()
        }
```
